The Australian National University

School of Computing

JHQ-GPU

Representation-Aware GPU ANN Search and Workload-Level Refinement Calibration

Rui Luo

Supervisor: Dr Mengxuan Zhang

A research thesis submitted for

COMP8800 — Advanced Computing Research Project

October 2026

Abstract

JHQ-GPU runs JHQ's primary/residual search on a GPU. It combines index construction, packed scanning, lane refinement and workload-level calibration. Cartesian splitting replaces 256 LUT entries with $16+16$, giving a storage ratio $32/256 = 1/8$ with the same real-arithmetic score. Calibration compares sampled top-k IDs with a larger-budget reference, without GT or a learned predictor.

On six RTX 5090 datasets, two-group LUTs lead in 22/24 cells; the largest index holds 17.78 million vectors at 20.44 GiB resident memory. Full-pool calibration meets a 10^{-3} held-out recall-loss target relative to the best tested budget in 12/12 cells, with maximum loss 0.0006; smaller samples expose workload-dependent quality risk. At recall 0.97 and batch 1,000, JHQ leads in three of four datasets and is near parity with IVF-RaBitQ on arXiv. OpenAI runs use TF32; FP32 checks retain favourable comparisons under a TF32-trained cache. At batch 8 and the same recall, CAGRA fp32 leads on all four datasets. These results locate the execution, quality and capacity trade-offs of GPU JHQ.

Acknowledgements

I thank Dr Mengxuan Zhang for supervising this project and acknowledge the JHQ authors and open-source ANN/GPU software cited throughout this thesis.

Research and AI assistance. This thesis extends joint JHQ-GPU work with Mengxuan Zhang [21] and builds on JHQ [16]. OpenAI Codex assisted with substantive drafting, revision, LaTeX, analysis and code/log checks [30]; ChatGPT supplied frameworks and reviews [29]; Anthropic Claude assisted with implementation, experiments and review [4]. Selected prompts and usage details are available in the AI-use record. The author is responsible for the submitted claims.

Contents

Abstract i

Acknowledgements i

Notation and Abbreviations vi

1 Introduction 1
- 1.1 Context and Practical Objectives . . . 1
- 1.2 Problem Definition and Scope . . . 1
- 1.3 Three Linked Technical Challenges . . . 1
- 1.4 Research Questions and Falsifiable Claims . . . 2
- 1.5 Research Contributions . . . 2

2 Background and Related Work 4
- 2.1 ANN Objective and Metrics . . . 4
- 2.2 IVF, PQ, ADC and Refinement . . . 4
- 2.3 JHQ Representation and Assumptions . . . 5
- 2.4 GPU Execution Concepts . . . 6
- 2.5 Graph Search on GPUs . . . 6
- 2.6 Quantised Search and Fast Tables . . . 6
- 2.7 Research Gap and Design Choices . . . 6

3 Research Design and Measurement 8
- 3.1 Research Questions and Comparisons . . . 8
- 3.2 Datasets, Queries, Ground Truth and Splits . . . 8
- 3.3 Timing, Variance and Memory Accounting . . . 9
- 3.4 Baselines and Frontier Construction . . . 10
- 3.5 Experimental Records and Reproducibility . . . 11
- 3.6 Evidence Strength and Threat Controls . . . 11

4 GPU Index Design and Construction 12
- 4.1 End-to-End Build and Search Boundaries . . . 12
- 4.2 Training, Rotation and Codebook Generation . . . 12
- 4.3 IVF Assignment, IDs and Physical Memory Layout . . . 14
- 4.4 Resident Storage, Workspace and Index Lifecycle . . . 16
- 4.5 Construction Cost and Tested Scale . . . 17
- 4.6 Index Representation and Safety Contracts . . . 18

5 Representation-Aware GPU Search 20
- 5.1 Query Path and Four Distinct Ranking Objects . . . 20
- 5.2 Exact Cartesian LUT Factorisation . . . 21
- 5.3 Packed Codes and Candidate-Parallel Memory Access . . . 23
- 5.4 Exact Global Primary Top-c via Multi-Block Scanning . . . 23
- 5.5 Shared Complete-Key Thresholds and Execution Scheduling . . . 26
- 5.6 Lane-Cooperative Residual Scoring and Final Top-k . . . 29
- 5.7 IVF Probe Selection as an Independent Primitive . . . 30

6 Workload-Level Budget Calibration 32
- 6.1 Why Fixed Refinement Budgets Can Be Wasteful . . . 32
- 6.2 GT-Free Sampled Output Agreement . . . 32
- 6.3 Budget Search, Nestedness and Tie Safety . . . 34
- 6.4 Budget Baselines and Their Questions . . . 36
- 6.5 Hold-out Risk, Calibration Cost and Payback . . . 36

6.6 Policy Failure Modes and Applicability . . . 37

7 Experimental Evaluation **38**
7.1 Experimental Protocol . . . 38
7.2 Modern GPU ANN Recall–QPS Frontiers . . . 38
7.3 Cartesian LUT, Packed Loads and Lane Refinement . . . 41
7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation . . . 43
7.5 Held-out Budget Quality, Throughput and Payback . . . 44
7.6 Batch Regimes, G Scheduling and Workload Skew . . . 47
7.7 Cold Construction, Resident Memory and Tested Scale . . . 49
7.8 Numerical Robustness, Anomalies and Recall Attribution . . . 51

8 Discussion and Future Work **53**
8.1 Three Cross-Layer Insights . . . 53
8.2 Two Deep Negative-Result Case Studies . . . 53
8.3 Threats to Internal and External Validity . . . 54
8.4 Testable Research Directions . . . 55
8.5 Conditional Deployment Recommendation . . . 55

9 Conclusion **56**
9.1 Answers to the Four Research Questions . . . 56
9.2 Closing Principle and Outlook . . . 56

Bibliography **57**

A Reproducibility and Experiment Manifest **60**
A.1 Source, Environment and Trained States . . . 60
A.2 Measurement Contracts . . . 60
A.3 Budget Harness Contract . . . 60
A.4 Supplementary Evidence . . . 60
A.5 Strict Selection, Small Batches and Artifact Access . . . 61

B Supplementary Results and Correctness Checks **62**
B.1 Parameter Coverage . . . 62
B.2 Correctness and Numerical Evidence . . . 62
B.3 Secondary Results and Evidence Status . . . 63
B.4 Construction Capacities and Precision Anchors . . . 63
B.5 Artifact Access . . . 64

List of Figures

1.1 Problem, mechanism and evidence . . . 2
2.1 Candidate and precision cascade . . . 4
3.1 Host and device timing boundaries . . . 10
4.1 Build and query architecture . . . 12
4.2 List-ordered physical index layout . . . 15
5.1 Query stages and selection boundaries . . . 20
5.2 Packed candidate-parallel access . . . 23
5.3 Local coverage and truncated pairwise merging . . . 25
5.4 Shared complete-key filtering . . . 28
5.5 Sixteen-lane residual refinement . . . 29
6.1 Work affected by the refinement budget . . . 32
6.2 Workload-level calibration and evaluation . . . 33
6.3 Nested and non-nested candidate pools . . . 35
7.1 E1 recall–throughput frontiers . . . 39
7.2 TF32 and FP32 precision frontiers . . . 40
7.3 Small-batch competitive regions . . . 41
7.4 Cartesian lookup-table grouping . . . 42
7.5 Packed access and refinement controls . . . 42
7.6 Held-out risk across sample sizes . . . 46
7.7 Finite-pool risk prediction and observation . . . 47
7.8 Direct cumulative calibrated-reuse time . . . 48
7.9 Batch-dependent execution regimes . . . 48
7.10 Recorded construction costs . . . 50
7.11 Resident-memory decomposition . . . 50
7.12 Large-dataset search frontiers . . . 52
8.1 Mailman and rank-oriented coding . . . 54

List of Tables

1.1 Technical contributions and evidence boundaries . . . 3
2.1 Design alternatives and research gap . . . 7
3.1 Research questions and experimental controls . . . 8
3.2 Dataset and training configuration . . . 9
3.3 Principal E1 tuning dimensions . . . 11
4.1 Per-vector storage model . . . 16
4.2 Analytical row-array occupancy by construction phase . . . 17
4.3 Original-input FP64 recoding audit . . . 18
4.4 Representation contracts and audit coverage . . . 19
5.1 Query-stage selection contracts . . . 21
5.2 Cartesian lookup-table trade-offs . . . 22
6.1 Budget controls and their roles . . . 36
7.1 Cross-system comparison at recall 0.97 . . . 38
7.2 FP32 sensitivity at matched configurations . . . 39
7.3 Small-batch matched-recall ratios . . . 40
7.4 Refinement gains at 128 probes . . . 43
7.5 Strict computed-score selection checks . . . 43
7.6 Probe-selection throughput ratios . . . 44
7.7 Full-pool budget calibration . . . 45
7.8 Sampled-budget held-out quality . . . 45
7.9 Sampled-calibration economics . . . 46
7.10 Direct costs for 500 reuse batches . . . 47
7.11 Integrated execution and scheduling gains . . . 49
7.12 Cold construction and resident memory . . . 49
7.13 E1 allocation ranges . . . 51
8.1 Threats to validity . . . 55
A.1 Component and run provenance . . . 60
B.1 Exact E1 configuration counts . . . 62
B.2 Validation coverage . . . 63
B.3 Independent construction-buffer capacities . . . 63
B.4 Same-session precision anchors . . . 63

Notation and Abbreviations

Symbol	Meaning
N, d	Number of database vectors and original dimension
M, D_s	Number of primary subspaces and dimensions per subspace
$a_{\text{add}}, a_{\text{in}}, a_{\text{assign}}$	Configured add, staged input, and independently allocated assignment rows
n_b, S_{stage}	Pipeline buffer count and byte budget per input/rotated array
Π	Square orthogonal transform; $q' = \Pi q$
B, B_r	Primary bits per subspace and residual bits per coordinate
$\rho(p), o_\ell$	Input ID at physical position p, and start of IVF list ℓ
k	Number of returned neighbours
n_c, c	Routed candidates and primary survivors receiving refinement
α	Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	Sample count; allowed missing-reference slots
$C_\alpha, \mathcal{R}_\alpha$	Valid survivors; returned IDs at budget α
$\alpha_{\text{ref}}, B_{\text{reuse}}$	Reference refinement budget and number of subsequent reuse batches
g_{LUT}	Number of groups in a factorised lookup table
g_{scan}	Number of scan groups per query; zero labels the one-block path
$C_{\text{buf}}, T_{\text{blk}}$	Buffer capacity; threads per block
$n_{\text{LUT}}(g_{\text{LUT}})$	LUT entries per primary subspace
$\kappa(D, p)$	Packed full key for bounds
ℓ_{ref}	Lanes cooperating on one survivor
D_{raw}	$\|q - x\|_2^2$
D_{P}	$\|q' - \widehat{p}\|_2^2$
D_{H}	$\|q' - \widehat{p} - \widehat{r}\|_2^2 + \|q'\|_2^2$

Abbreviation	Meaning
ANN	Approximate nearest neighbour
ADC	Asymmetric distance computation
IVF	Inverted file
JHQ	Johnson–Lindenstrauss enhanced hierarchical quantisation
LUT	Lookup table
PQ	Product quantisation
QPS	Queries per second
GT	Ground truth
FP32	IEEE binary32; GEMM mode stated separately
TF32	Reduced-precision Tensor Core inputs; FP32 accumulation
CV	$\text{sd}(n_c)/\,\text{mean}(n_c)$

1 Introduction

1.1 Context and Practical Objectives

CAGRA navigates a graph; compressed IVF scans chosen lists [18, 28, 33]. Stella shows why compact storage matters (section 7.7):

$$S_{\text{raw}} = 67.81\,\text{GiB} > S_{\text{GPU}} = \frac{32607}{1024}\,\text{GiB} > S_{\text{resident}} = 20.44\,\text{GiB}.$$

JHQ screens with primary codes and refines survivors with residuals [16]. We vary execution and survivor count on the same trained index, measuring recall, speed, memory and construction cost, including losses.

1.2 Problem Definition and Scope

For $\mathcal{X} \subset \mathbb{R}^d$, query q and selected IVF lists $\mathcal{L}(q)$, the valid-pool pipeline is

$$\begin{aligned} q' &= \Pi q, & U(q) &= \bigcup_{\ell \in \mathcal{L}(q)} \text{list}_\ell, \\ c &= \max\{k, \lceil \alpha k \rceil\}, & C_\alpha(q) &= \text{Top}^{\text{P}}_{\min(c,|U(q)|)}\, U(q), \\ & & \mathcal{R}_\alpha(q) &= \text{Top}^{\text{H}}_k\, C_\alpha(q). \end{aligned}$$

Top selection uses each computed score and its tie rule. The refined score includes a query-only constant (section 2.3); GT measures raw-neighbour recovery.

GPU construction, layout and search retain JHQ's square rotation and Cartesian primary/residual codes. One unlabelled-agreement multiplier is reused on a stable workload; it is neither per-query learning nor an absolute-recall certificate.

E1 tests squared-L2 top-10 on one RTX 5090, four datasets and batch 1,000. Six datasets support component/budget tests; T5 adds batches 8/32 (section 7.2.5). Search stays GPU-resident with host preparation/transfers. Updates, host paging, multi-GPU and billion-scale search are untested.

1.3 Three Linked Technical Challenges

Figure 1.1 connects three challenges to the mechanisms and evidence used to address them.

First, the primary codebook has useful mathematical structure that a direct GPU port can overlook. Each eight-bit subspace code indexes 256 distances. Its Cartesian coordinates permit an exact 16+16 factorisation, but more aggressive splitting adds lookups and arithmetic. The challenge is to exploit separability without assuming that the smallest table gives the fastest scan. Candidate-parallel word access and residual lane cooperation also need different data mappings.

Second, local quotas can lose global winners. For disjoint groups U_1, U_2 and $c = 4$, consider

$$\begin{aligned} \text{Top}_4(U) &\subseteq U_1, \\ \left|\text{Top}_4(U) \cap \left(\text{Top}_2(U_1) \cup \text{Top}_2(U_2)\right)\right| &= 2 < 4. \end{aligned}$$

Each group therefore needs enough coverage before merging. Shared bounds reduce work, while batch/skew scheduling handles the overhead. Selecting IVF lists remains separate.

Third, reference agreement cannot fix reference error. A $k = 1$ counterexample is

$$\mathcal{R}_\alpha = \mathcal{R}_{\text{ref}} = \{a\}, \quad \mathcal{T}_1 = \{b\}, \quad a \neq b \implies D_S(\alpha) = 0, \quad \text{Recall@1} = 0 \qquad (S = 1).$$

We therefore test quality and calibration-plus-reuse cost together.

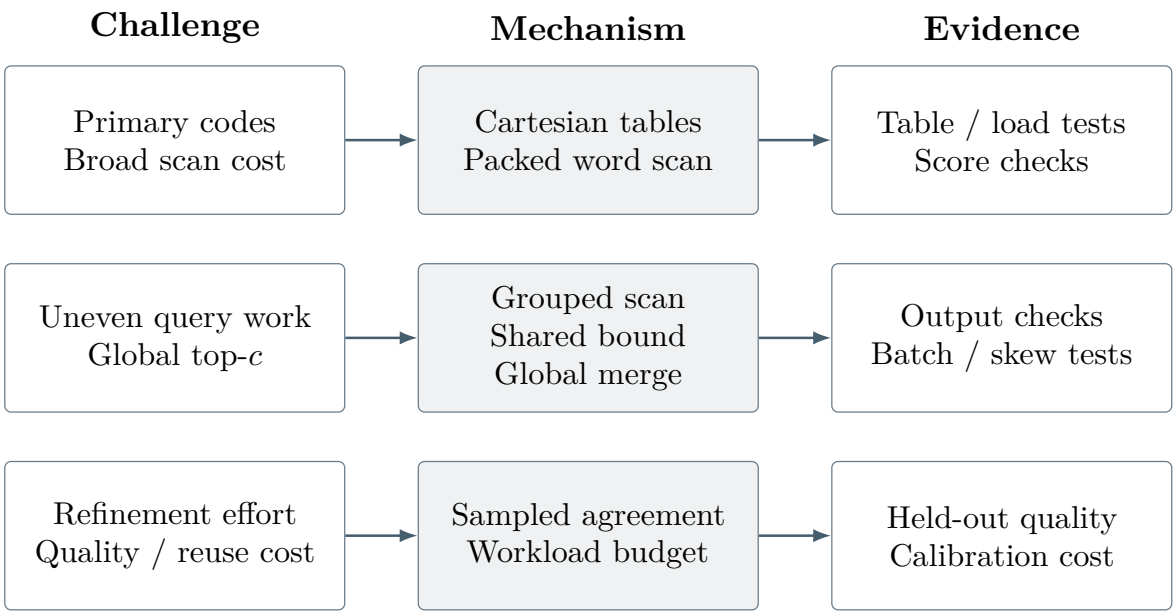


Figure 1.1. Problem, mechanism and evidence. The arrows organise research questions; they do not imply that independently measured speedups multiply or that an unlabelled agreement rule guarantees GT recall.

1.4 Research Questions and Falsifiable Claims

RQ1: Construction and capacity. How can JHQ be built on a GPU, and what cost, resident capacity and build peak result (section 7.7)?

RQ2: Correct and efficient execution. Which execution choices speed up search while preserving score/tie contracts (sections 7.3, 7.4 and 7.6)?

RQ3: Budget quality and total cost. Can unlabelled calibration preserve held-out quality and repay its cost (section 7.5)? The GT-labelled oracle is an evaluation control.

RQ4: Competitive operating regions. Under host-to-host timing, where does JHQ lead at matched recall? The answer uses independently tuned E1 frontiers and explicit memory definitions (section 7.2). Int8 graph competitors, high-recall reversals and absent coverage are part of the answer.

Chapter 3 turns these questions into controlled comparisons. Together they separate what was built, whether its execution is correct, what effort a workload requires, and whether the resulting system has useful operating regions.

1.5 Research Contributions

Table 1.1 states three linked project contributions and their inherited foundations. The core contribution is the adaptation and evaluation of a particular representation–execution–budget combination. JHQ, Cartesian quantisation, IVF, bitonic/radix selection, merging and CUDA Graph execution retain their prior-art status.

Table 1.1. Technical contributions and their evidence boundaries.

	Inherited foundation	Project contribution	Evidence
C1	JHQ rotation, codebooks and hierarchy	GPU training/encoding, bounded batch construction and list-ordered packed representation	Search of 17.78M vectors at 20.44 GiB resident; repeated train+add 10.54–10.65 s; sampled construction peak 22.58 GiB
C2	Cartesian additivity and established GPU primitives	Integrated factorised scoring, complete-key grouped retention/bounds, lane refinement and execution scheduling	Two-group LUTs lead in 22/24 cells; scheduled execution 2.727× at batch 8; explicit correctness scope
C3	Candidate expansion and output-agreement ideas	Workload-level budget rule, conditional selection analysis, held-out controls and cost/risk evaluation	FULL meets the relative-loss target in 12/12 cells; smaller samples expose risk; four cells directly measure calibrated reuse

2 Background and Related Work

2.1 ANN Objective and Metrics

For stable-ID database $\mathcal{X} \subset \mathbb{R}^d$, exact neighbours minimise

$$D_{\text{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

Recall@10 measures overlap with stored GT, including its tie choices (equation (3.1)), rather than a worst-case distance ratio.

We measure QPS, batch latency, build cost and memory. Batch timing does not measure concurrent-request tails; memory figures have different boundaries. Chapter 3 defines them.

Finite frontiers, qualifying thresholds and interpolation follow section 3.4. Uncovered targets remain unknown.

2.2 IVF, PQ, ADC and Refinement

IVF groups vectors into coarse-centroid lists. A query chooses n_{probe} lists, producing a routed universe $U(q)$ of n_c candidates. Visiting fewer lists reduces access work but can exclude a true neighbour before any fine scoring. Choosing lists and ranking database vectors are separate steps.

PQ learns one codebook per coordinate subspace [17]. ADC keeps q unquantised:

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

A query table replaces coordinate scoring with lookups. Products across subspaces do not imply Cartesian structure within a learned subcodebook; Chapter 5 needs the latter.

The residual definitions differ:

$$r_{\text{IVF-PQ}} = x - \mu_{\ell(x)}, \qquad r_{\text{JHQ}} = \Pi x - \widehat{p}(x).$$

JHQ reconstruction must not add a centroid the encoder never removed. Candidate expansion/refinement is established [10, 18].

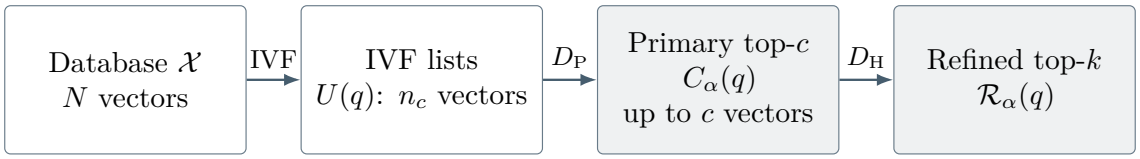
With $|U(q)| \geq k$ and $c = \max\{k, \lceil \alpha k \rceil\}$,

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

For GT set $\mathcal{T}_k$, set inclusion gives

$$|\mathcal{R}_\alpha \cap \mathcal{T}_k| \leq |C_\alpha \cap \mathcal{T}_k| \leq |U \cap \mathcal{T}_k|.$$

Refinement cannot recover excluded IDs. Isolating each loss still needs fixed-universe counterfactual scoring.



Query $q' = \Pi q$ stays FP32.
Primary codes score the broad scan.

Residual $r = \Pi x - \widehat{p}$.
Survivors decode $\widehat{p} + \widehat{r}$.

Figure 2.1. Candidate and precision cascade. IVF selects lists; primary scoring selects vectors; only survivors read residuals. The hierarchy is inherited. This diagram defines logical sets and score fidelity, not measured recall losses or GPU kernel boundaries.

$$W_{\text{primary}} = O(n_c M) \text{ lookups}, \qquad W_{\text{refine}} = O(cd) \text{ coordinates}.$$

Probes choose U; α caps survivors within U. Latency also pays for access and selection.

2.3 JHQ Representation and Assumptions

2.3.1 What is inherited

JHQ combines rotation and a primary/residual hierarchy [16]. For square $\Pi \in \mathbb{R}^{d\times d}$,

$$\begin{aligned} \Pi^\top \Pi = I \Longrightarrow \|\Pi(q-x)\|^2 \\ = (q-x)^\top \Pi^\top \Pi (q-x) = \|q-x\|^2. \end{aligned} \tag{2.4}$$

Dimension remains d. This is a real-arithmetic identity; quantisation, FP32 orthogonality and GEMM error are separate checks.

Partition y into M subspaces of dimension $D_s = d/M$. With B bits per primary subspace, $K = 2^B$ labels are formed from $L = 2^{B/D_s}$ scalar levels, requiring the implemented Cartesian parameters to make L an integer. Writing Φ for the standard normal CDF, the constructor uses

$$a_i = \sigma \Phi^{-1}\left(\frac{i-1/2}{L}\right), \quad i = 1,\dots,L, \qquad \mathcal{C}_m = \{a_1,\dots,a_L\}^{D_s}. \tag{2.5}$$

At the evaluated $B = D_s = 8$, $L = 2$ and $\mathcal{C}_m = \{-a, +a\}^8$, with $a \simeq 0.67449\sigma$. Each byte records eight coordinate signs. The implementation estimates σ from vector energy; unit-normalised inputs motivate the scale $1/\sqrt{d}$. This representation is prior art, not an invention of the GPU execution layer.

Residuals are $r = y - \widehat{p}$, giving reconstruction $\widehat{y} = \widehat{p} + \widehat{r}$. The evaluated codes use $d/8$ primary bytes and d residual bytes per vector, before metadata. Refining fewer rows reduces reads, not stored residual size.

2.3.2 Model assumptions and score semantics

Random rotation can spread coordinate energy, motivating a shared Gaussian-inspired scalar model. It does not force an arbitrary realised database to have independent, zero-mean Gaussian coordinates. For a fixed rotation and empirical database mean μ, the transformed mean is $\Pi\mu$ and $\|\Pi\mu\| = \|\mu\|$. Averaging over random rotations is different from centring one fixed corpus. Even a uniformly random point on a sphere has dependent coordinates through its fixed norm. Consequently, isometry is an algebraic guarantee; the distributional approximation is a modelling assumption. Chapter 8's centred-coding pilot examines one practical consequence of that distinction.

The inherited primary levels are Gaussian mid-quantiles. For $X \sim \mathcal{N}(0,\sigma^2)$ and a zero threshold,

$$\begin{aligned} a_{\text{MSE}} &= \arg\min_a \mathbb{E}[(X-a)^2 \mid X > 0] \\ &= \mathbb{E}[X \mid X > 0] = \sigma\sqrt{2/\pi} \simeq 0.79788\sigma. \end{aligned}$$

For fixed signs and positive scale, however,

$$D_{\text{P}} = \|q'\|^2 + da^2 - 2a\langle q', s\rangle, \qquad s \in \{-1,+1\}^d. \tag{2.6}$$

For fixed signs,

$$a > 0 \Longrightarrow \arg\min_x D_{\text{P}}(q,x) = \arg\max_x \langle q', s(x)\rangle.$$

Scale still changes residuals; separability is not a quantiser-optimality claim.

The scores use different vectors:

$$D_{\text{raw}} = \|q - x\|^2, \qquad D_{\text{P}} = \|q' - \widehat{p}\|^2.$$

$$D_{\text{H}} = \|q' - \widehat{p} - \widehat{r}\|^2 + \|q'\|^2. \tag{2.7}$$

Removing $\|q'\|^2$ preserves order, not raw-vector accuracy.

2.4 GPU Execution Concepts

CUDA warps have 32 threads; blocks share limited registers/memory [25]. JHQ parallelises candidates in scans and coordinates in refinement. Extra blocks also duplicate state.

Faiss uses warp selection [18]; bitonic, radix and merge trade state for work. AIR/GridSelect cover global selection [38]. These are inherited tools.

2.5 Graph Search on GPUs

SONG parallelises GPU graph search [39]; CAGRA tunes graph connectivity and search width [28]. Both fp32 and quantised CAGRA belong in the comparison.

BANG trades host-graph transfers for capacity [37]; JHQ keeps its compressed index on-device.

2.6 Quantised Search and Fast Tables

2.6.1 Learning a representation versus exploiting its structure

Learned OPQ codebooks may lose Cartesian separability [13]. ScaNN weights inner-product error [15]. Lower reconstruction error alone need not preserve nearest-neighbour ranking.

2.6.2 Small lookup tables have substantial prior art

Fast Scan and Quicker ADC shape CPU register tables [1, 2]. JHQ uses GPU Cartesian splitting, with FP32/tie checks.

RaBitQ supplies a specific randomised estimator [11]; extra bits trade space for accuracy [12]. Its bounds do not transfer to JHQ. GPU IVF-RaBitQ is a fused-search baseline [33].

2.6.3 Refinement effort and adaptive stopping

Faiss/cuVS expose refinement effort [7, 10]; alternatives use GT tuning [9], learned stopping [19] or recall targets [6]. Our rule uses workload-level reference agreement.

2.7 Research Gap and Design Choices

Table 2.1 compares the alternatives by the constraint they address. The literature already provides GPU graph search, compressed distance computation, register-oriented lookup, fast selection and adaptive effort. A useful contribution must therefore specify the representation contract, the integration decision and the evidence for it. “A GPU version” alone does not identify that contribution.

Table 2.1. Design alternatives and the thesis's narrower research gap. This is a mechanism comparison; measured rankings follow in Chapter 7.

Alternative	Useful structure	Principal trade-off	JHQ-GPU question
IVF-PQ / OPQ	Learned subspace codes; ADC	Distortion and training; arbitrary subcodebooks couple coordinates	What does an internally Cartesian primary code permit in GPU scoring?
Fast Scan / Quicker ADC	Small tables and SIMD lookup	Table precision, shape and ISA constraints	Does exact splitting repay extra GPU lookups?
SONG / CAGRA	Selective graph navigation	Dependent access, graph storage and construction	Where does a resident hierarchical scan compete?
BANG	Compressed GPU scoring with host graph	Capacity versus communication	What can complete compressed residency achieve?
IVF-RaBitQ	GPU code construction and fused search	Bit budget, estimator and probe effort	Which recall regions justify the residual hierarchy?
Adaptive effort	Refine ratios, autotuning, learned or declarative stopping	Labels, reference fidelity, prediction or calibration cost	Can unlabelled workload agreement select useful effort at an acceptable risk and total cost?

We hold JHQ's representation fixed to study three linked choices: correct GPU execution, useful scan parallelism and refinement quality versus total cost.

3 Research Design and Measurement

Paired execution, held-out calibration and system frontiers use different controls, defined here. Methods follow in Chapters 4–6 and results in Chapter 7.

3.1 Research Questions and Comparisons

Table 3.1 defines the comparison unit and what would challenge each research claim. A *cell* fixes a dataset, index configuration and workload setting; arms are alternative implementations or budgets within that cell. For paired execution measurements, the index state, queries, probe count, refinement budget and scoring representation are held fixed unless the named treatment changes them. The recorded paired ratio is $Q_{\text{treatment}}/Q_{\text{control}}$. Across J stated cells, equal weighting gives

$$G = \exp\left(\frac{1}{J}\sum_{j=1}^{J}\log\frac{Q_{\text{treatment},j}}{Q_{\text{control},j}}\right).$$

Table 3.1. Research questions as testable comparisons. A capacity observation has a different control from a paired execution intervention.

Question	Intervention / comparison	Held fixed or accounted for	Measure and counter-evidence
RQ1: build and capacity	GPU construction; cached versus cold lifecycle; compressed versus raw storage	N, d, representation, routing and timed construction boundary	Train/add time and resident bytes; failed builds or model–allocation gaps delimit feasibility
RQ2: execution	LUT grouping, word access, refinement lanes, grouped scan, shared bound and probe selection	Paired state, workload and score; reference output semantics	Wall QPS, stage time and output checks; slowdowns or non-tie mismatches challenge the mechanism
RQ3: budget	Sampled / full-pool rule versus fixed 64, 100, 200, enumeration and GT oracle	One index and route; disjoint calibration and held-out query positions	Held-out recall loss, selected-arm QPS and calibration cost; threshold crossings challenge reliability
RQ4: operating region	Independently tuned JHQ, CAGRA, IVF-PQ and IVF-RaBitQ	Exported vectors, GT, k, device and host-to-host boundary; memory reported separately	Matched-recall QPS and coverage; stronger competitors at other recall targets challenge broad superiority

Ablations measure speed changes; cache/occupancy claims need counters. Complete systems require separate comparisons (section 7.6).

E1 retains extended grids, losses and rejected alternatives. Agreement measures reference stability; held-out GT measures raw-neighbour recall.

3.2 Datasets, Queries, Ground Truth and Splits

Table 3.2 records the six exported datasets used by the newer budget, LUT and scale campaigns. The poster's E1 comparison uses the first four. Vogue contains fashion-image embeddings; arXiv contains abstract embeddings; the two OpenAI datasets use Qdrant's DBpedia exports [5, 31, 32, 34]. BGE uses the Italian Wikipedia configuration, and Stella uses the TREC24 BioGen corpus export [35, 36]. Sizes describe the measured exports after preparation, rather than the current upstream cards.

Table 3.2. Data and JHQ training configuration. n_q is the actual query count; $n_t = 39n_{\text{list}}$ is the requested coarse-training sample size. E1 uses the first four rows. All rows use squared L2 and $k = 10$.

Dataset	N	d	n_q	n_{list}	n_t
Vogue	932,328	768	1,000	4,096	159,744
arXiv	2,253,000	768	1,000	8,192	319,488
OpenAI-1536	999,000	1,536	1,000	8,192	319,488
OpenAI-3072	999,000	3,072	1,000	8,192	319,488
BGE-M3	10,091,524	1,024	1,000	32,768	1,277,952
Stella	17,776,615	1,024	999	32,768	1,277,952

For OpenAI/BGE/Stella, preparation filters non-finite rows and applies

$$x \leftarrow x/\|x\| \quad (\|x\| > 10^{-12}).$$

Seed 42 samples queries without replacement. Stella uses 999 corpus queries, not its 65-query benchmark. Relevance labels are absent. The OpenAI exports share the sampling rule; checks support row alignment on all queries and 10,299 base rows, without original-ID joins (sections 7.2 and 7.2.3).

All methods in a cell read the same query and GT files. Ground truth is computed by exhaustive squared-L2 search over exported FP32 vectors; the streamed converter maintains top-k across database tiles. Evaluation uses stored GT order, including its floating-point tie choices. With $\mathcal{T}_k(q)$ the first k GT IDs and $\mathcal{R}_k(q)$ the returned ID set,

$$\text{Recall}\,@k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

GT rows are 100 wide for Vogue/arXiv and 20 wide for the other four; only ten entries are scored. Duplicate returned IDs do not earn repeated credit. Range, duplicate and repeat-call checks are recorded separately.

JHQ uses $M = d/8$, $B = B_r = 8$, eight coarse iterations and 100,000 residual-training rows. Sweeps reload models and re-encode rows; cold builds refit. Training uses database rows.

$$\mathcal{Q}_{\text{cal}} = \{q_i : i \bmod 2 = 0\}, \qquad \mathcal{Q}_{\text{hold}} = \{q_i : i \bmod 2 = 1\},$$

$$(|\mathcal{Q}_{\text{cal}}|, |\mathcal{Q}_{\text{hold}}|) = \begin{cases} (500, 499), & \text{Stella}, \\ (500, 500), & \text{otherwise}. \end{cases}$$

Seeded subsamples of sizes 32, 64 and 128 are drawn 64 times per cell; FULL uses all 500 calibration queries. Labels assess the oracle and holdout; agreement uses outputs (sections A.3 and 6.2).

3.3 Timing, Variance and Memory Accounting

The measured interval is the host search call:

$$t_{\text{call}} = t_{\text{return}} - t_{\text{entry}}, \quad \begin{cases} \text{entry: FP32 queries in host memory,} \\ \text{return: host IDs/scores after stream completion.} \end{cases}$$

It includes staging, transfers, query work and int8 query quantisation; fitting/build are excluded (figure 3.1). Resource handles and output buffers are reused. Output/API differences remain: JHQ pageable/C++; CAGRA/PQ pinned/Python; RaBitQ pinned/C++. Small margins include these costs.

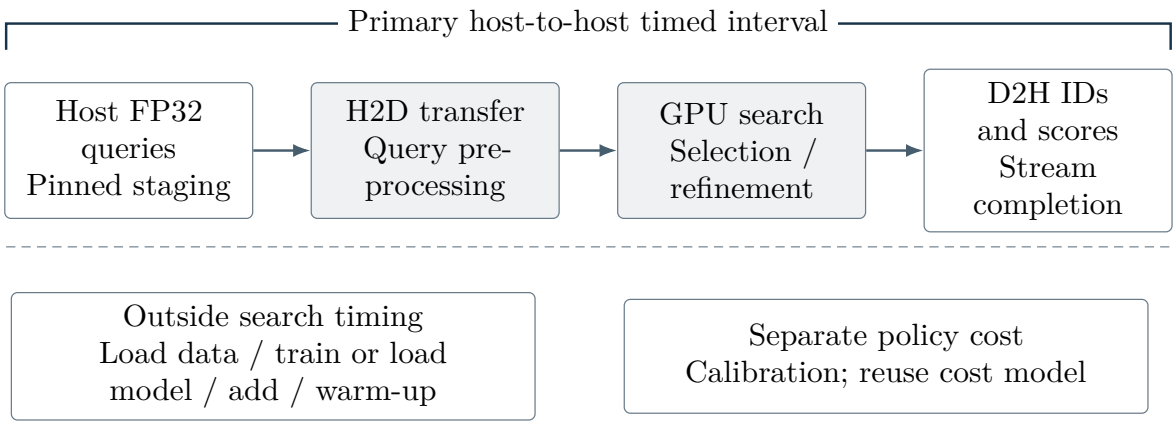


Figure 3.1. Timing boundaries. The host interval includes transfers and query-specific computation. Device-stage profiles explain parts of that interval; separate profiling and synchronisation can change execution, so their sum is not an interchangeable end-to-end latency.

With t in seconds,

$$\bar{t}_r = \frac{1}{m_r}\sum_{i=1}^{m_r} t_{ri}, \qquad Q_r = \frac{b}{\bar{t}_r}, \qquad Q = \mathrm{median}_r\, Q_r. \tag{3.2}$$

E1 uses batch 1,000, five rounds and normally 24 calls. JHQ warms once per variant/round; baselines warm three times initially. Slow baseline points use fewer calls. Warm-up excludes JIT/capture; percentiles describe batch calls, not concurrent requests.

Slow valid calls stay. Mean-call and median-call rates differ; for an illustrative $(1, 1, 10)$ ms sequence,

$$\bar{t} = 4\,\mathrm{ms} \neq \mathrm{median}(t) = 1\,\mathrm{ms}, \qquad b/\bar{t} \neq b/\,\mathrm{median}(t).$$

Shared indexes/pools are not independent training replications.

Runs use RTX 5090/32,607 MiB, driver 595.71.05, CUDA 13.0 and cuVS 26.08.01. OpenMP uses 16 threads within 25 CPUs; E1 records 575 W. Locks do not fix clocks/load. JHQ JIT-compiles compute_80 PTX [27]; library builds and device profiles remain separate.

Resident totals include models, workspace and context; CAGRA's index deltas differ from JHQ's context-subtracted totals (sections 4.4 and 7.7). Later pools and build scratch prevent peak-matched comparison. Build peaks use 20 ms sampling; train/add excludes input loading and host setup.

Supplementary evidence boundary. Supplementary sessions are separate (section A.4). Precision changes and cached-model reuse follow sections 4.2 and 7.2.3; historical baseline ratios remain cross-session.

3.4 Baselines and Frontier Construction

The competing families cover graph traversal (CAGRA fp32/int8), product-quantised IVF and GPU IVF-RaBitQ [17, 28, 33]. FP32 GPU brute force provides an exhaustive-search reference. Each family sweeps its own implementation parameters (table 3.3); the recorded index configurations can differ in storage and reconstruction fidelity. Consequently the main frontier measures achievable speed at a quality target within these grids. A separate memory account accompanies it.

Table 3.3. Principal E1 tuning dimensions. Exact per-point parameters and coverage extensions remain in the raw records. The grid is finite and differs between methods.

Family	Recorded search dimensions
JHQ	Probes and α; initial probes 64/128/256 and 0.5/1/2 times the workload budget; coverage extensions reach probe 1,024 and larger/smaller budgets
CAGRA fp32/int8	Graph degree 32/64; search width 1/2/4/8; intermediate top-k capacity 32–512
IVF-PQ	Eight-bit subcodes; 192/384/768-byte codes at $d = 768$, 384/768/1,536 at higher dimensions; fp16/fp32 lookup settings; probes 16–512
IVF-RaBitQ	4/8/9 bits per dimension, supported search modes, probes 16–1,024; additional 4,096-list OpenAI indexes
GPU brute force	Exhaustive FP32 squared-L2 search, common host boundary

Dominance and the measured frontier are

$$a \succ b \iff R_a \geq R_b,\ Q_a \geq Q_b,\ (R_a, Q_a) \neq (R_b, Q_b).$$

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

No qualifying measured point means undefined. For adjacent frontier points,

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

The parser snaps within 5×10^{-4}, otherwise interpolates without extrapolation. T5 retains qualifying measured endpoints below a method's minimum recall. Both estimators are reported near ties; storage is unmatched.

Frontiers exclude failed/invalid outputs, not repeat-ID flags alone. Isolated reruns replace contaminated builds; skipped Stella batch-1,000 files add no points (section A.2). Int8 CAGRA remains included (figure 7.1); older ADC/poster plots keep their own scope.

3.5 Experimental Records and Reproducibility

Records name version, settings, timing and outputs. Paired tests share trained states; state or precision changes define a new family. Missing historical executable/model fingerprints limit exact reproduction (appendix A).

3.6 Evidence Strength and Threat Controls

Correctness is score/tie-specific; raw recall also includes routing/reconstruction loss. Sections 6.2, 6.3 and 8.3 give the sampling, ordering and measurement limits.

4 GPU Index Design and Construction

JHQ is trained and encoded into bounded-memory, list-ordered GPU arrays [16]; chapter 5 and section 7.7 cover search and measured cost.

4.1 End-to-End Build and Search Boundaries

Training fits models; add encodes/list-orders rows; search uses the finished index. A cache hit skips fitting, not encoding. Cold build and cached search therefore time different work.

The host builds samples, rotation and primary codebooks. GPUs rotate, fit IVF/residual models, then encode bounded batches into list order.

Models store rotation, codebooks and centroids; database state stores offsets, IDs, codes and corrections. Reused query allocations are overwritten; graphs reference them.

One successful `add` fixes the snapshot and positions. Another add is rejected; updates require rebuilding. Mutable/transactional publication is unimplemented.

Host queries return host IDs/scores; equation (5.20) and section 3.3 define their meaning and timing.

4.2 Training, Rotation and Codebook Generation

4.2.1 Transform and sampling

Host LAPACK QR turns a seeded Gaussian matrix into square Π. Device row-major rotation is

$$Y_t = X_t \Pi^\mathsf{T}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^\mathsf{T}\Pi = I. \tag{4.1}$$

Dimension stays d; C0 checks finite-precision QR/GEMM error (section 4.6).

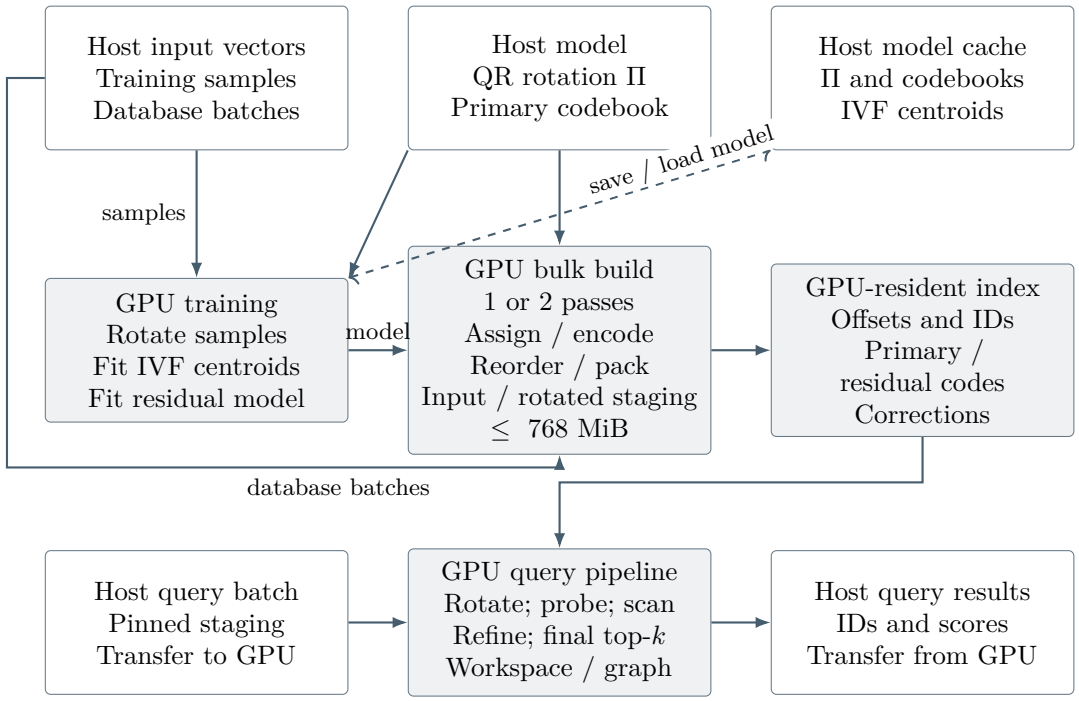


Figure 4.1. Implemented build and query boundaries. Host work includes sampling, orthogonal-matrix construction and analytical primary-codebook generation; the evaluated configuration fits IVF and residual models on the GPU. The cache stores trained state, whereas the completed database representation resides on the device. Dashed arrows: model reuse. The staging bound covers raw/rotated floats at source defaults, excluding other scratch.

Strided primary/IVF samples use $n_t = 39n_{\text{list}}$: 159,744 Vogue rows, 319,488 arXiv/OpenAI rows and 1,277,952 BGE/Stella rows. This matches FAISS's warning threshold, not a quality guarantee [8]. Residual fitting separately uses 100,000 rows.

An earlier twelve-cell Vogue/OpenAI-3072 sweep ($B_r = 4/8$, probes 32/128/256) fixed primary/IVF models. At 25 Lloyd iterations, 100K rather than all rows lost at most 0.0018 recall (eleven losses, one tie); intermediate sizes were non-monotone. This supports bounded training, under that older protocol.

4.2.2 Why retain the analytical primary model?

Sample scale and Cartesian levels are

$$\sigma^2 = \frac{1}{n_t d}\sum_{i=0}^{n_t-1}\|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

$$B = D_s = 8 \Longrightarrow \mathcal{C}_m = \{-a, +a\}^8, \quad a = 0.674\sigma.$$

Eight signs occupy one byte. This JHQ structure supports $O(d)$ encoding and split LUTs [16] (section 5.2); arbitrary Lloyd updates can destroy it.

For fixed sign vector $s \in \{-1, +1\}^d$,

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

$$a > 0 \Longrightarrow \arg\min_s \|q' - as\|^2 = \arg\max_s \langle q', s\rangle.$$

This real-arithmetic order is Gaussian-independent. Residuals still change with a; $a = 0$ and FP32 ties need separate treatment.

Historical Vogue ($M = 96$, $B_r = 4$, 128 probes) gave primary recall 0.6618/0.6342 and full recall 0.9452/0.9409 for earlier/analytical builds. Residual training also changed, so this is not a codebook-only effect. Chapter 7 tests the full frontier.

4.2.3 Coarse assignment as filter and refine

IVF starts from spaced rotated rows and runs eight iterations, ranking $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$. Under supported dimensions, default int8 filtering resolves ambiguous assignments in FP32.

Lemma 4.1 (Assignment with certified score intervals)**.**
Let $\widetilde{y}, \widetilde{\mu}_\ell$ be dequantised approximations, $e = y - \widetilde{y}$ and $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$. Keeping centroid norms exact, the approximate score $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle\widetilde{y}, \widetilde{\mu}_\ell\rangle$ has real-arithmetic error bounded by

$$E_\ell = 2(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|). \tag{4.4}$$

Given valid intervals $[L_\ell, U_\ell]$ containing every target score, discarding only $L_\ell > \min_j U_j$ and exactly resolving the remaining scores preserves the minimum and its specified tie rule.

Proof.
Using $e = y - \widetilde{y}$ and $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$,

$$\begin{aligned}\langle y, \mu_\ell\rangle - \langle\widetilde{y}, \widetilde{\mu}_\ell\rangle &= \langle y, f_\ell\rangle + \langle e, \widetilde{\mu}_\ell\rangle,\\ |s_\ell - \widetilde{s}_\ell| &\le 2(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|) = E_\ell.\end{aligned}$$

For $j^* \in \arg\min_j U_j$,

$$L_\ell > U_{j^*} \Longrightarrow s_\ell \ge L_\ell > U_{j^*} \ge s_{j^*}.$$

Such ℓ cannot minimise the score. Equality stays eligible, preserving ties. □

FP32 headroom is conservative, not a bitwise certificate without accumulation-error bounds. FP16 comparisons differ on 0/14,838 Vogue, 6/31,748 BGE and 3/16,359 Stella tail assignments. Cold logs do not identify the executed precision.

4.2.4 Residual fitting and the iteration budget

After IVF, residuals remain $r = \Pi x - \widehat{p}(x)$. Each subspace fits $K_r = 2^{B_r}$ sorted scalar levels shared across coordinates, using GPU sorting/Lloyd updates capped at 2,000.

Historical 100K samples gained 0.0063–0.0076 recall from 25→2,000 updates. Tail bins and BGE/Stella changes persisted, so reaching the cap does not establish convergence or contradict exact Lloyd descent. At 20,000 updates, historical Stella all-row/100K recall was 0.9921/0.9920.

4.2.5 Construction work and observed imbalance

$$W_{\mathrm{QR}} = O(d^3), \qquad W_{\mathrm{rotate}} = O(n_t d^2).$$

Direct coarse assignment costs $O(n_t n_{\mathrm{list}} d)$ per iteration.

$$W_{\mathrm{primary}} = O(MKD_s), \qquad W_{\mathrm{residual\ extraction}} = O(n_r d),$$

followed by sorting and scalar updates. These describe work, rather than predict latency. Synchronised phase diagnostics place about 83% of BGE/Stella training in IVF k-means (about 2.97 s). They use one add buffer and forced synchronisation, separately from the normal overlapped timings in section 7.7.

List-size imbalance enters the scan rule in section 5.5. The individual contributions of eight IVF iterations, spaced seeds and finite sampling to that imbalance were not isolated.

Arithmetic used in the measured implementation. The evaluated implementation (v69) stores FP32 matrices but enables cuBLAS TF32 tensor-core math for $d \geq 1{,}536$. In its compiled faithful branch, a null override falls through to the dimension-based default. This affects training/add/query rotation and centroid GEMMs on both OpenAI datasets; an environment override cannot disable that branch. Thus FP32 storage does not imply full FP32 multiplication. The supplementary FP32 variant (v72) corrects this switch. Its sensitivity experiment reuses the same TF32-trained cache and changes add/query arithmetic, rather than retraining the model in FP32 (section 7.2.3).

4.3 IVF Assignment, IDs and Physical Memory Layout

4.3.1 From database IDs to list-ordered positions

An input row has identifier $i \in \{0, \ldots, N-1\}$. Coarse assignment places its transformed vector in a list according to the computed centroid scores. The mathematical assignment target is

$$a(i) = \operatorname*{arg\,min}_{0 \leq \ell < n_{\mathrm{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

Its implementation uses the selected numerical assignment path. Encoding the primary and residual representations does not subtract $\mu_{a(i)}$; list membership restricts the future search universe independently of those reconstructions.

The builder sorts assignments together with a sequence of input IDs. Let $\rho(p)$ be the input ID at sorted physical position p and let o_ℓ be the start of list ℓ. The completed arrays satisfy

$$0 = o_0 \leq o_1 \leq \cdots \leq o_{n_{\mathrm{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

Lower-bound searches give offsets, with

$$o_\ell = o_{\ell+1} \iff \text{list } \ell \text{ is empty.}$$

Positions are stable within a snapshot, not across builds. All arrays use the same $\rho(p)$.

IDs map $p \mapsto \rho(p)$. Two-pass add temporarily builds ρ^{-1} for scatter. Primary ties use positions; final ties use database IDs.

4.3.2 Exact addresses and representation widths

For primary byte $Z[m, p]$,

$$\text{offset}_{\text{byte}}(Z[m, p]) = mN + p.$$

Final packing produces

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \text{addr}_{\text{byte}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{4.7}$$

Here $j = 0, \ldots, M/4 - 1$. word-layout build $\implies M \bmod 4 = 0$; other M are rejected. A separate byte-layout compilation retains the byte representation. Each packed array occupies NM bytes, but repacking temporarily holds both old and new copies, costing $2NM$. The one-pass builder temporarily gathers a vector-major $[N, M]$ primary array, transposes it, and then repacks; each obsolete copy is freed. The two-pass builder scatters directly into the subspace-major array before the same final repack. The search-time lane mapping is discussed in Section 5.3.

Residual codes remain row-major, with $b_r = \lceil dB_r/8 \rceil$ bytes per vector. In the eight-bit configuration, coordinate t of row p is at byte offset $pb_r + t$. In the four-bit path it is in byte $pb_r + \lfloor t/2 \rfloor$, with even coordinates in the low nibble and odd coordinates in the high nibble. A four-byte correction per physical row stores

$$\gamma[p] = 2\langle \widehat{p}(x_{\rho(p)}), \widehat{r}(x_{\rho(p)}) \rangle. \tag{4.8}$$

Its value must move with that row's codes during gathering or scattering. Residual access then reconstructs the same object that primary screening identified, even though the two code arrays use different layouts.

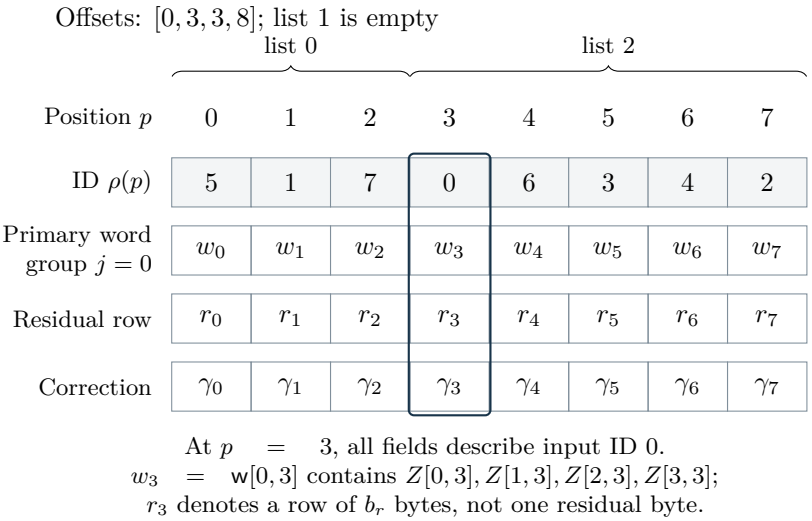


Figure 4.2. One common physical position joins IDs, packed primary words, residual rows and correction factors. This illustrative eight-row index has offsets $[0, 3, 3, 8]$, including an empty list. One word packs four subspaces of one vector; values/list sizes are schematic.

Primary codes scan in list order; residual rows are fetched only for survivors. Retaining raw FP32 vectors on the device would provide exact reranking but consume $4Nd$ additional bytes and change the reported search design. We refine compressed reconstructions; Chapter 7 measures the resulting quality.

4.4 Resident Storage, Workspace and Index Lifecycle

4.4.1 Stored bytes and allocated bytes

With one-byte primary subspace codes, resident row and offset bytes are

$$S_{\text{rows}} = N\left(M + \left\lceil \frac{dB_r}{8} \right\rceil + 8\right), \qquad S_{\text{offsets}} = 4(n_{\text{list}} + 1). \tag{4.9}$$

$S_{\text{ID}} + S_{\text{correction}} = 4N + 4N = 8N$ bytes. At $M = d/8$, $B_r = 8$, codes take $9d/8$ bytes, 12.5% above SQ8. Graph-free does not imply smaller than int8.

At $d_g = 32$, CAGRA row bytes are $d + 4d_g$. Stella's JHQ payload (19.20 GiB) nearly equals CAGRA int8's 19,656 MiB, with different scopes. JHQ is graph-free and changes residual reads via α, not stored bytes.

Basic model storage is

$$S_{\text{model}} = 4d^2 + 4MKD_s + 4MK_r + 4n_{\text{list}}(d+1) \quad \text{bytes.}$$

Derived arrays are additional. Workspace holds two $[b_{\text{cap}}, d]$ query matrices, centroid scores, LUTs, probes and survivors. Optional residual LUTs add $4b_{\text{cap}}dK_r$ bytes; defaults score directly. Group capacity remains about 93.8 MiB at 128 probes, even without grouped execution (equation (5.13)).

Input is caller-owned. Separate staging/order storage is

$$S_{\text{pinned}} = 4n_b a_{\text{in}} d, \qquad S_{\text{order}} = S_{\text{inverse}} = 4N.$$

Assignment scratch is freed before gather (equations (4.10) and (4.13)).

4.4.2 Reuse, recovery and invalidation

Caches save models but omit code version/full-data hashing from the key. Changed code/data need invalidation and provenance. Sweeps share states; cold builds refit. Similar recall does not prove identical models.

Cache hits still pay QR, warm-up, model upload and add. Stella's 6.67 s add includes initial workspace, excluding schedule calibration and search warm-up; it is not restart time.

α changes need no retraining. Workspace grows before capture; changes to batch, survivors, probes, lanes, groups or bounds recapture the graph. Calibration may bypass capture. Index-owned buffers are reused, then released.

Table 4.1. Per-vector bytes. JHQ includes codes, ID and correction; SQ8 is vector codes alone; CAGRA int8 models vector codes plus 32 four-byte neighbours. Offsets, model parameters, workspace and other runtime allocations are excluded. These components are not complete process totals.

d	Primary	Residual	ID + corr.	JHQ	SQ8	CAGRA int8	FP32
768	96	768	8	872	768	896	3,072
1,024	128	1,024	8	1,160	1,024	1,152	4,096
1,536	192	1,536	8	1,736	1,536	1,664	6,144
3,072	384	3,072	8	3,464	3,072	3,200	12,288

4.5 Construction Cost and Tested Scale

4.5.1 Bounded input batches and two-pass construction

Whole-database raw/rotated storage is $4Nd + 4Nd = 8Nd$ bytes. Bounded staging avoids it. For n_b pipeline buffers, configured add rows a_{add} and per-array byte budget S_{stage}, the source stages

$$a_{\text{in}} = \max\left\{1, \min\left(a_{\text{add}}, \left\lfloor \frac{S_{\text{stage}}}{4d} \right\rfloor\right)\right\}, \qquad S_{\text{float}} = 8n_b a_{\text{in}} d. \tag{4.10}$$

$$4d \le S_{\text{stage}}, \quad n_b = 3,\ S_{\text{stage}} = 128\,\text{MiB} \Longrightarrow S_{\text{float}} \le 2n_b S_{\text{stage}} = 768\,\text{MiB}.$$

Assignment/encoder scratch is additional.

One-pass add gathers encoded input-order arrays, keeping both copies briefly. Two-pass add assigns/sorts first, then re-reads and scatters encodings directly. It saves simultaneous storage at the cost of another rotation/read; raw assignment against back-rotated centroids uses orthogonality.

4.5.2 Phase occupancy and the process peak

The one-pass estimate is

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes}. \tag{4.11}$$

Its ten live arrays sum to

$$3NM + 2Nb_r + 20N = N(3M + 2b_r + 20).$$

The added 1 GiB is heuristic headroom, not a whole-process peak bound.

Final two-pass repack counts IDs, residuals, corrections and two primary copies:

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.12}$$

Whole-build peak also considers earlier pipeline/inverse scratch, model, sort, context and later workspace allocations.

Table 4.2 quantifies the compressed-copy saving. Stella's cold log selects two passes and prints 41.6 GiB for equation (4.11), against 30.6 GiB available at the tail. The separately sampled resident process totals, including context and query workspace, are 11.93 GiB for BGE and 20.44 GiB for Stella (section 4.4). The supplementary 20 ms device samples observe a 26.08 GiB BGE peak near the start of add, and a 22.58 GiB Stella peak during repack. These are sampled device totals including context; shorter-lived peaks can be missed.

Assignment-product capacity uses an independent configured row count a_{assign}, selected by `JHQ_ASSIGN_BATCH` or the source default: 32,768 for at least 8,192 lists, otherwise 8,192.

$$S_{\text{dots}} = 4n_b n_{\text{list}} a_{\text{assign}} \quad \text{bytes}. \tag{4.13}$$

Allocation capacity need not shrink with a_{in} or the rows in a particular call. Table B.3 checks four defaults: the 1,536/3,072-dimensional cases allocate 3 GiB, despite different staging rows. BGE/Stella allocate $3 \times 4 = 12$ GiB near add entry, before the 23.05 GiB gather lifetime. The

Table 4.2. Analytical row-array occupancy (GiB), $d = 1{,}024$, $M = 128$, $B_r = 8$. BGE uses one pass; Stella uses two. The one-pass Stella column is counterfactual capacity analysis. Pipeline, model, offsets and runtime costs are excluded from these phase values.

Dataset	Final row payload	One-pass gather	Two-pass repack
BGE	10.90	23.05	12.11
Stella	19.20	40.59	21.32

peak is

$$S_{\text{peak}} = \max_t \sum_{A \text{ live at } t} |A|,$$

not the sum of those separate phase totals. Stella's smaller repack plus model/context explains its lower observed peak.

4.5.3 Construction measurements

4.6 Index Representation and Safety Contracts

A 32-bit $n_t d$ overflow at 2^{31} zeroed primary scale; a version-blind cache reloaded it. Chunked double-precision norms fixed the fault. Successful routing alone is insufficient validation.

4.6.1 Completed representation evidence

The 13 October pilot exported full cached layouts for Vogue (932,328 rows), arXiv (2,253,000) and OpenAI-3072 (999,000).

Codebooks passed separability, scalar-level and sorting checks. An FP64 loop checked $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$ on every exported row; maximum absolute error was 1.28×10^{-7}–2.77×10^{-7}. Sampled GPU-returned top-10 scores have maximum relative errors from 2.38×10^{-7} to 4.85×10^{-5}, using $|s_{\text{GPU}} - s_{64}|/s_{64}$ with the query-norm constant included. These are checks of the exported representation, not re-encoding from the original input vectors.

Across seven workloads and 896 query instances, the pilot additionally checked the proposed residual bounds against FP64 reconstructed distances for 72,800,590 routed candidate instances, with zero reported violations. The same population supports the FP64 composite-score identity check. GPU-returned-score checks cover only top-10 slots and skip IDs absent from the routed map, leaving output membership incompletely checked.

Encoder, primary/residual training and construction sources match the evaluated implementation after namespace normalisation and removal of the export method. This source correspondence covers the shared path, leaving later model identity and large-dataset two-pass construction to separate checks.

4.6.2 Independent input and placement audit

The supplementary exporter audit uses the shared construction path on Vogue, arXiv, OpenAI-3072 and Stella. Offsets are monotone with endpoints 0 and N; IDs form a permutation of every database row in all four. FP64 products of the stored rotation give maximum entrywise $\|\Pi\Pi^{\mathsf{T}} - I\|$ errors of 3.96–4.34×10^{-7}. Sampled norm checks use FP64 rotation, rather than the GPU GEMM, and therefore measure the stored matrix's geometry. Table 4.3 summarises sampled recoding.

Table 4.3. Original-input FP64 recoding audit. Flips count coordinate decisions; the final column is the largest residual-boundary distance among mismatches. Structural checks cover all rows; recoding is sampled.

Dataset	Rows	Orth. error	Primary flips	Residual flips	Max margin
Vogue	200,000	4.11e-07	14	5,409	1.12e-07
arXiv	100,000	4.11e-07	1	2,515	7.31e-08
OpenAI-3072	20,000	3.96e-07	5,745	1,646,806	2.67e-05
Stella	100,000	4.34e-07	16	3,064	1.03e-07

At 768/1,024 dimensions, recoding mismatches fall within 1.12×10^{-7} of boundaries. OpenAI-3072 has 2.68% residual mismatches, up to 2.67×10^{-5} from boundaries, consistent with TF32. Conditioning on stored primary digits avoids double-counting flips.

ID-joined builds have identical codes on three smaller databases, corrections within 8.94×10^{-8}, and 1/1/40 Vogue/arXiv/OpenAI assignment differences. Stella checks only two-pass construction. Missing exported centroids leave membership unverified.

4.6.3 Contract coverage and remaining checks

Required conditions are

$$M \mid d, \quad D_s \mid B, \quad M \bmod 4 = 0.$$

Inputs must be finite; counts/sentinels valid. IDs use signed 32-bit fields; address products use wider arithmetic.

T4 checks selection (section 7.4.1); exports leave membership, empty lists and adversarial tails uncertified.

Table 4.4. Representation contracts after the pilot and supplementary input audit. Checks separate geometry, arithmetic and row placement.

Boundary	Required property	Evidence / remaining check
C0: geometry	Common transform with bounded floating-point isometry error	Stored-matrix orthogonality checked on four datasets; TF32 GPU arithmetic distinguished
C1: codebooks / correction	Cartesian levels, sorted residual codebooks, correction matches decoded codes	Pilot checks completed on three databases; all-row correction errors in section 4.6
C1: input encoding	Stored codes encode the intended original input row	FP64 original-input recoding on four samples; boundary and TF32 discrepancies quantified
C1: placement	Offsets cover rows; IDs form a permutation; all fields follow it	All-row offsets/ID checks; three ID-joined path comparisons; centroid membership unverified
Execution	Valid indexing, synchronisation and score-order contracts	Archived query and sanitizer checks; finite coverage, detailed in section 7.4

5 Representation-Aware GPU Search

This chapter develops GPU scoring/selection for JHQ's transform and code hierarchy [16]; chapters 6 and 7 give measurements and budget choice.

5.1 Query Path and Four Distinct Ranking Objects

Each query follows $q' = \Pi q$, IVF routing, primary screening and residual refinement, returning k IDs/scores. Figure 5.1 distinguishes the three runtime selection decisions; the ground-truth ordering is an external evaluation object.

Four quantities must be kept separate:

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

Here μ_i is an IVF centroid in transformed coordinates, $\widehat{p}(x)$ is the primary reconstruction and $\widehat{r}(x)$ is the decoded residual. The runtime refined score includes a query-only constant, as derived from the implemented correction in Section 5.6. It preserves the ordering of reconstructed distances for a fixed query, but neither it nor the primary score is the original-vector distance.

Routing defines the visited universe U of n_c candidate occurrences; primary screening retains up to $c = \max\{k, \lceil \alpha k \rceil\}$ for residual scoring. Table 5.1 records each stage's inputs, reads and costs; sections 5.2 to 5.7 develop their implementations.

The ordering and exactness columns of table 5.1 attach each claim to its score and tie rule. Chapter 7 tests survivor sets, ordered outputs and raw-space recall separately.

Table 5.1 gives the stage contracts. The E1/budget harness explicitly selects 16 refinement lanes and scan policy -1 (scheduling among 0/2/4/8), versus library defaults of scalar refinement and policy 0. Shared bounds are enabled but used only at $g_{\mathrm{scan}} > 1$; the harness calibrates scheduling with 512 database pseudoqueries, a step the library does not perform automatically. Component overrides and implementation versions are in table A.1.

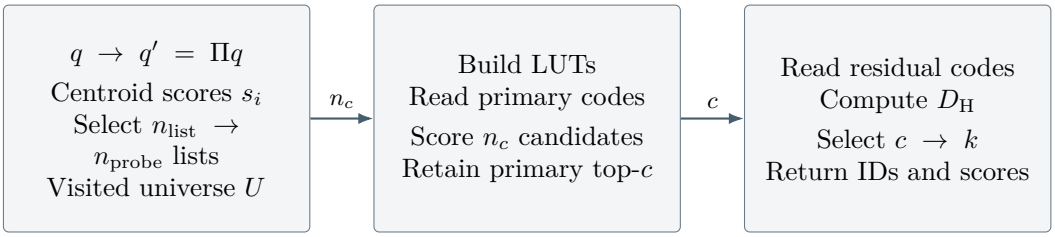


Figure 5.1. Query execution and its separate selection boundaries. Routing selects lists, primary screening selects vector candidates, and final selection ranks refined survivors. Raw-space ground truth is used for evaluation; it is not an input to these search decisions.

Table 5.1. Query-stage contracts. $c_v = \min(c, n_c)$ counts valid primary survivors. Exactness refers to the stated computed scores; routing and quantised reconstruction determine raw-space approximation.

Stage	Input → output	Data read	Main work	Order / exactness
Transform	$q \to q'$	Query; transform	Apply Π	Transform (precision in section 7.2.3)
Probe	$n_{\text{list}} \to n_{\text{probe}}$ lists	FP32 centroids; list lengths	Score; radix; rank count	(s_i, i); exact probe selection
LUT	$q' \to$ tables	Primary codebook	$MD_s 2^{8/g_{\text{LUT}}}$ terms	Real-arithmetic factorisation; FP32 rounding
Primary, $g_{\text{scan}} = 0$	$n_c \to c_v$	Packed primary codes; LUT	$O(n_c M)$ scoring; compaction	Exact distance-order selection; arbitrary cutoff ties
Primary, $g_{\text{scan}} \geq 1$	$n_c \to c_v$	Same primary representation	Score; compact; merge	(D_{P}, p); exact under stated premises
Refine	$c_v \to c_v$ scores	Residual codes; correction	$O(c_v d)$	Computed D_{H}; lane-dependent FP32 association
Final	$c_v \to k$	Refined scores; IDs	$O(kc)$ slot scans	$(D_{\text{H}}, \text{ID})$; exact with k finite valid entries

5.2 Exact Cartesian LUT Factorisation

Primary ADC uses FP32 q' and stored reconstructions [16, 17]:

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\text{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

The 256-entry table can shrink because each subcodebook is Cartesian.

For $K = 256$, $D_s = 8$, the analytical codebook gives

$$\begin{aligned} a &= \sigma\Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8, \\ D_{\text{P}}(q, x) &= \|q'\|_2^2 + da^2 - 2a \sum_{j=0}^{d-1} s_j(x) q'_j. \end{aligned} \tag{5.3}$$

Here Φ is the standard normal CDF; a byte stores eight signs, higher bits first. Thus

$$a > 0 \quad \Longrightarrow \quad \arg\min_x D_{\text{P}}(q, x) = \arg\max_x \sum_j s_j(x) q'_j.$$

Four-/eight-way Cartesian splits are also exact in real arithmetic. Learned replacements require compatibility checks.

5.2.1 Factorisation without changing the quantiser

Write $z = 16h + l$, with $h, l \in \{0, \ldots, 15\}$ and $\mathcal{C}_m[z] = (A_m[h], B_m[l])$. Then

$$T_m^{\text{hi}}[h] = \|q'_{m,\text{hi}} - A_m[h]\|_2^2, \qquad T_m^{\text{lo}}[l] = \|q'_{m,\text{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

Coordinate separation gives

$$T_m[z] = T_m^{\text{hi}}[z \gg 4] + T_m^{\text{lo}}[z \,\&\, 15]. \tag{5.5}$$

Storage falls to $16 + 16 = 32$ entries; codes, candidates and real-arithmetic scores stay fixed.

The builder slices codewords $16h$ and l; `verify_split_lut` checks that each shared nibble has a shared coordinate slice. An arbitrary learned 256-centroid PQ subcodebook need

not satisfy this condition: dividing its code labels into nibbles does not factor its centroid coordinates.

The same-candidate FP32 audit in section 7.3 measures numerical and ranking differences between table forms, including changes after identical primary survivor sets.

5.2.2 Grouping, construction work and resource cost

The table-grouping ablation generalises the split to $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ contiguous coordinate groups whose bit fields match the Cartesian digits. For eight bits and $D_s = 8$, each group has $2^{8/g_{\mathrm{LUT}}}$ entries; the total entries and FP32 storage per query are

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}} 2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4M n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes.} \tag{5.6}$$

Independent entry construction evaluates $D_s 2^{8/g_{\mathrm{LUT}}}$ coordinate contributions per subspace. Two groups therefore reduce table storage eightfold and coordinate work sixteenfold relative to independently constructing every full-table entry. These are table-level cost ratios, not predicted application speedups.

The table is built in global workspace and staged cooperatively when its complete shared-memory footprint fits the queried per-block opt-in limit $S_{\max}$. For buffer capacity C_{buf}, ordinary/grouped scan footprints are

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

For the evaluated paths (without optional fused-refinement/early-exit scratch),

$$\text{shared lookup} \iff S_{\mathrm{path}} \le S_{\max}.$$

In the table sweep, $c = 1{,}000$, block sizes are 512 or 1,024, and $C_{\mathrm{buf}} = 2{,}048$. Full tables span 96–384 KiB, versus 12–48 KiB for two groups. NVIDIA specifies a 99 KiB per-block limit for compute capability 12.0 [24]. With this limit, the full-table footprint exceeds capacity even at $M = 96$ (114,696 bytes in the query-centric path), whereas the largest two-group footprint is 65,560 bytes in the grouped path. Thus the source-level placement rule sends full tables to global memory and two-group tables to shared memory in these configurations. This placement is inferred from the launch rule and published limit. The measurement records do not include a device-attribute query.

Four and eight groups both store 16 entries per subspace but perform more lookups and combinations. Extraction, lookup dependencies, staging, synchronisation and residency jointly determine the measured trade-off. Multiple scan groups may stage the same table independently.

The $16 + 16$ representation is a nibble LUT for an eight-bit sign code. Small LUTs have precedents in Fast Scan, Quicker ADC and IVF-RaBitQ [1, 2, 33]; JHQ retains its own two-level score. Our contribution is their GPU integration with retention and refinement. Two groups lead in 22/24 measured cells; other margins are below 1% (section 7.3).

Table 5.2. Cartesian table trade-offs for an eight-bit, eight-coordinate primary subspace. Combining additions are the algebraic minimum for joining the looked-up values, excluding accumulation across subspaces; they are not measured instruction counts.

g_{LUT}	Entries	FP32 bytes	Lookups	Combining adds	Build terms
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

5.3 Packed Codes and Candidate-Parallel Memory Access

Neighbouring scan lanes process neighbouring candidates. Subspace-major storage makes code accesses adjacent; splitting shrinks the query LUT.

5.3.1 Address mapping and four-byte packing

For list ℓ, row r maps to

$$p = o_\ell + r, \qquad \text{offset}(Z[m,p]) = mN + p.$$

At fixed m, adjacent positions have adjacent bytes. Crossing lists can still cause a jump.

A word packs four subspaces of one candidate:

$$\mathsf{w}[j,p] = \sum_{t=0}^{3} 2^{8t} Z[4j+t,p], \qquad \text{offset}_{\text{bytes}}(\mathsf{w}[j,p]) = 4(jN+p). \tag{5.8}$$

For $0 \le t < 4$ and eight-bit Z,

$$(\mathsf{w}[j,p] \gg 8t)\ \&\ 255 = Z[4j+t,p].$$

The recovered nibbles index equation (5.5).

Construction repacks $[M,N]$ bytes into $[M/4,N]$ words, requiring $4 \mid M$. A `uint32_t` cast would pack consecutive candidates instead. Its code-byte costs are

$$R = W = NM, \qquad S_{\text{old+new}} = 2NM.$$

This one-time repack lies outside search timing.

5.3.2 What packed access saves, and what it preserves

One word load plus unpacking replaces four byte loads. A warp's logical window is $32 \times 4 = 128$ bytes; alignment/active lanes set transactions. Payload stays M bytes per candidate. Both arms are subspace-major, so transpose/cache effects are not isolated.

5.4 Exact Global Primary Top-c via Multi-Block Scanning

Groups scan in parallel and merge computed-primary survivors [18].

5.4.1 Candidate partition and deterministic ordering

For a fixed query, concatenate the selected inverted lists in probe order to obtain n_c candidate occurrences. Assume that each visited database row occurs once and that the index

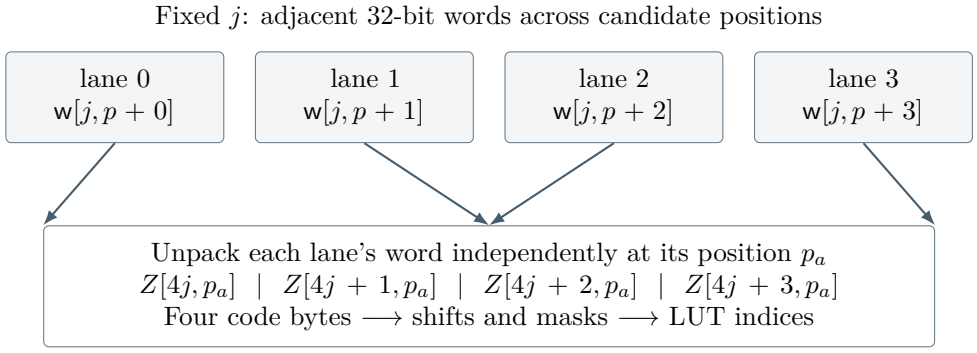


Figure 5.2. Packed candidate-parallel access, showing a subset of warp lanes. At fixed word group j, consecutive positions access adjacent four-byte words. Each word contains four subspace codes of one candidate. Unpacking preserves each code byte; LUT splitting happens afterwards. The diagram illustrates addresses, not a measured number of memory transactions.

stays fixed during search. For $g_{\text{scan}} \in \{1, 2, 4, 8\}$, block (q, j) owns the interval

$$U_j = \left[\left\lfloor \frac{jn_c}{g_{\text{scan}}} \right\rfloor, \left\lfloor \frac{(j+1)n_c}{g_{\text{scan}}} \right\rfloor\right), \qquad j = 0, \ldots, g_{\text{scan}} - 1. \tag{5.9}$$

Here an interval denotes the corresponding candidates, rather than their database IDs. The device computes its endpoints from the routed candidate count. Groups differ in size by at most one candidate; a long list can cross a group boundary. This balances candidate counts, whereas assigning equal numbers of lists need not do so. Each thread finds its first list through the probe-prefix offsets and advances its list cursor while scanning its interval. Residual codes are not read at this stage.

The selection key is

$$K_q(x) = \big(D_{\text{P}}(q, x), p(x)\big), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

where $p(x)$ is the physical row position in the fixed inverted-list storage. It is unique among the query's candidates and independent of the order of atomic appends. It is not the database ID used to break final-result ties. With non-NaN scores, this defines a total order on distinct candidates. Local selection and merging use this order. Partition independence also requires identical computed scores across paths.

5.4.2 Exact local retention and global coverage

$\text{Top}_c(V)$ contains the first $\min(c, |V|)$ keys, where $c = \max\{k, \lceil \alpha k \rceil\}$. Each group keeps up to c, since all global winners may lie there; a c/g_{scan} quota is unsafe.

Lemma 5.1 (Local-buffer retention)**.**
Let V_t be the candidates scored by a block through chunk t, and B_t its valid buffer after any compaction or filtering at that point. Assume fixed complete keys, sufficient append capacity and exact top-c compaction. Let β_t be the minimum of the complete shared bounds observed so far, including the initial top sentinel. Then

$$\text{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

With sharing disabled, the bound is the sentinel and the buffer contains the whole prefix top-c.

Proof.
The empty prefix is immediate. Put

$$E_t = \text{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\}.$$

On extending V_t to V_{t+1}, $\beta_{t+1} \preceq \beta_t$ and

$$\text{Top}_c(V_{t+1}) \cap V_t \subseteq \text{Top}_c(V_t).$$

Hence every old member of E_{t+1} was retained. A locally rejected new key has c earlier predecessors, so cannot enter the new prefix top-c. All other eligible arrivals fit by the capacity assumption. For any eligible winner x,

$$\#\{y \in B : K_q(y) \prec K_q(x)\} \le \#\{y \in V_{t+1} : K_q(y) \prec K_q(x)\} < c,$$

so exact compaction keeps x. Filtering removes only keys above an observed bound; their running minimum covers stale reads. These steps preserve the invariant inductively. □

Proposition 5.1 (Coverage and truncated merging)**.**
For fixed keys under a common total order, let U be the visited universe and V any retained subset. If $\mathrm{Top}_c(U) \subseteq V \subseteq U$, then $\mathrm{Top}_c(V) = \mathrm{Top}_c(U)$. Pairwise sorted merging with truncation to c preserves this result. In particular, for disjoint groups,

$$\mathrm{Top}_c\left(\bigcup_j U_j\right) = \mathrm{Top}_c\left(\bigcup_j \mathrm{Top}_c(U_j)\right). \tag{5.12}$$

The statement includes universes smaller than c.

Proof.
Every global winner is in V, and every other member of V follows the global winners. Selection from V therefore recovers them. For the independent-group special case, let x belong to the global top-c and to group U_j. Fewer than c candidates precede x globally, so fewer than c precede it within U_j. Consequently $x \in \mathrm{Top}_c(U_j)$, and the local union contains every global winner. Selecting its first c therefore recovers precisely the global winners in the same order. If the whole universe has fewer than c members, every member is retained.

For merging, the first c of two sorted lists are their union's top-c. A discarded candidate already has at least c predecessors within that union and cannot enter the top-c of any larger union. Induction over the merge tree establishes the claim. □

The CUDA implementation maintains a shared-memory buffer of C_{buf} key pairs, where C_{buf} is a power of two and $C_{\mathrm{buf}} \geq c + T_{\mathrm{blk}}$ for a scan block of T_{blk} threads. Bitonic compaction keeps up to c valid keys and sets the threshold to the last key once c exist. Before that point the threshold is a sentinel above every valid key. Between compactions, admission requires $K \prec \lambda_j$. The threshold has c witnesses, so remains an upper bound on the prefix cth key as candidates arrive.

Compaction leaves at most c entries; a chunk adds at most T_{blk}:

$$|B| + |\Delta V| \leq c + T_{\mathrm{blk}} \leq C_{\mathrm{buf}}.$$

This supplies lemma 5.1's capacity premise without thread quotas. With propositions 5.1 and 5.2, it preserves global winners through filtering and merging.

Materialising scores costs $O(n_c)$ writes/storage. Fused scoring compacts local buffers and emits $O(g_{\mathrm{scan}}c)$ pairs. Register selection is another option [18].

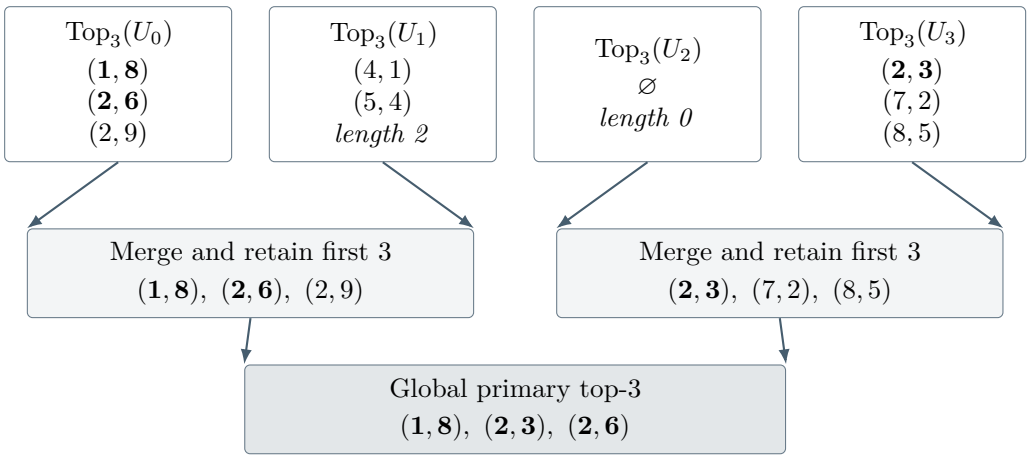


Figure 5.3. Local coverage and truncated pairwise merging, illustrated with $c = 3$ and $g_{\mathrm{scan}} = 4$. Entries are illustrative (D_{P}, p) keys; bold ones are global winners. Short/empty groups contribute their valid prefixes. Merges keep at most c in the same order.

5.4.3 Merge implementation, storage and boundary cases

At $g_{\text{scan}} > 1$, a 256-thread block merges each query over $\log_2 g_{\text{scan}}$ rounds. Each pair emits $\min(c, a + b)$ entries using Merge Path slices [14]. All comparisons use equation (5.10); block barriers separate rounds. Figure 5.3 illustrates the truncation invariant. The tree emits at most $c(g_{\text{scan}} - 1)$ key pairs across its internal nodes, in addition to the partition searches and synchronisation work.

The two global score/position workspaces have capacities $[b_{\text{cap}}, g_{\text{scan}}, c]$ and $[b_{\text{cap}}, g_{\text{scan}}/2, c]$ and alternate between rounds. With four-byte scores and positions, their combined payload is

$$S_{\text{pairs}} = 8 b_{\text{cap}} c \left(g_{\text{scan}} + \frac{g_{\text{scan}}}{2} \right) \quad \text{bytes}, \qquad g_{\text{scan}} > 1. \tag{5.13}$$

Counts add $4 b_{\text{cap}} g_{\text{scan}}$ bytes and provisioned bounds add $8 b_{\text{cap}}$, beyond survivor/other workspace. Allocation grows before capture and stays reserved for reuse. Merging stages lengths, not all keys, in shared memory.

Valid counts exclude padding:

$$K_{\text{sort pad}} = (+\infty, \text{INT_MAX}), \qquad K_{\text{output pad}} = (+\infty, -1).$$

Valid positions are below INT_MAX; empty groups have count zero. Refinement skips negative positions, and merges exclude them. At $g_{\text{scan}} = 1$, survivors go directly to output.

$g_{\text{scan}} = 0$ uses one block per query with distance-exact selection and free cutoff-tie IDs. Lossy `K_LOCAL=4` requires `JHQ_EXACT_TOPCK=0`; paper-faithful builds disable it.

For nonempty scanned set V, let $h = \min(c, |V|)$ and d_h be its hth score. Distance-only compaction yields

$$\{x : d_x < d_h\} \subseteq B \subseteq \{x : d_x \leq d_h\}, \qquad |B| = h.$$

A rejection has c witnesses with no larger score; only tied identities remain free. T4 checks this contract and grouped full-key order by CPU sorting GPU-produced scores (section 7.4.1).

5.5 Shared Complete-Key Thresholds and Execution Scheduling

Full local top-c retention establishes correctness, but can produce $g_{\text{scan}} c$ intermediate survivors for only c global outputs. The shared-bound component reduces this excess selection work. The scheduler then chooses how much scan parallelism to expose. Both operate at a fixed refinement budget; choosing that budget is the separate problem in Chapter 6.

5.5.1 A complete-key upper bound with explicit witnesses

Let $K_{(c)}$ be the global cth key when $|U| \geq c$. If a group holds c distinct, fully scored candidates, the largest key among those witnesses, τ_j, satisfies $K_{(c)} \preceq \tau_j$. There are already c candidates no later than τ_j in the global universe. The shared bound is initialised to a top sentinel and updated by

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

No bound is published from fewer than c valid candidates. If the entire universe has fewer than c, no group can publish such a bound and all valid candidates remain eligible.

Proposition 5.2 (Safe rejection by a witnessed bound)**.**
Assume fixed keys, publications backed by c distinct candidates, and reads that return either the initial sentinel or one complete published key. Rejecting a candidate only when $K_q(x) \succ \tau$ cannot remove a global top-c candidate, irrespective of publication timing.

Proof.
A publication has a set W_τ of c distinct witnesses:

$$\max_{x \in W_\tau} K_q(x) \preceq \tau \quad \Longrightarrow \quad K_{(c)} \preceq \tau.$$

Sentinel reads and minima of published keys obey the same inequality. Thus

$$x \in \mathrm{Top}_c(U) \implies K_q(x) \preceq K_{(c)} \preceq \tau_{\mathrm{seen}} \implies x \text{ is not rejected.}$$

Stale reads only loosen the bound. Publications made after earlier safe rejections remain valid: any c retained distinct candidates still supply c witnesses, whether or not they are the best c of their group. □

A shared bound may discard local winners while preserving proposition 5.1's global coverage. Admission is

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\mathrm{seen}}, \tag{5.15}$$

Here λ_j is the local compaction threshold and τ_{seen} the last shared key read. Equality with the shared bound stays eligible.

5.5.2 Publication, filtering and implementation limits

For nonnegative FP32 scores, canonical positive zero and valid nonnegative positions,

$$\kappa(D, p) = 2^{32}\,\mathrm{uintbits}_{32}(D) + p. \tag{5.16}$$

Non-NaN float bits are increasing on this domain, so

$$(D_1, p_1) \prec (D_2, p_2) \iff \kappa(D_1, p_1) < \kappa(D_2, p_2).$$

Infinity follows finite scores; the all-one sentinel follows valid keys. Invalid positions cannot publish.

One thread polls per chunk. After compaction, c valid keys permit a 64-bit `atomicMin` publication. Final publication/binary search trims the sorted prefix; exact merging handles stale-bound extras. Searches reset bounds in-stream and merge after scanning, without inter-group waiting.

Aligned `ld.volatile.global.u64`/`atom.global.min.u64` satisfy PTX single-copy atomicity for compatible strong accesses [26]. CUDA 13.0 `ptxas`, `sm_120` corroborates this (section A.4); driver-JIT SASS was not captured. C++ `volatile` alone supplies no such argument [25].

The bound follows full scoring (figure 5.4), saving buffer/merge work while adding polling/atomics. It can slow search (section 7.4).

5.5.3 Choosing parallelism without changing the budget

Grouping launches bg_{scan} blocks, adding parallelism, staging, selection and merging. Smaller quotas need a certificate/fallback. Concentrated pilot survivors motivated witnessed bounds (section 7.4).

Let T_{blk} be scan threads per block, H the number of SMs times the maximum resident threads per SM, and $v = \mathrm{sd}(n_c)/\,\mathrm{mean}(n_c)$ the candidate-count coefficient of variation. Its

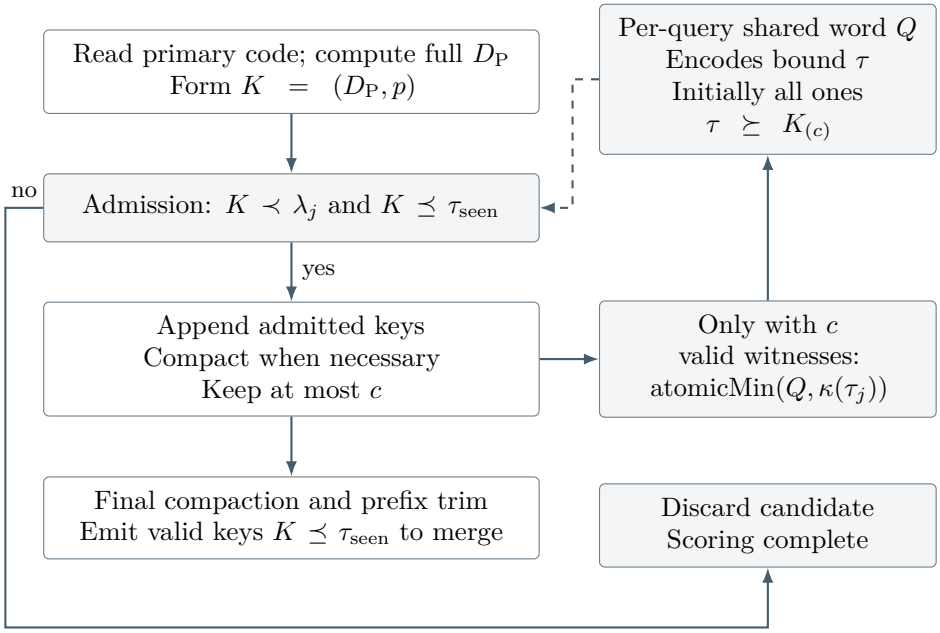


Figure 5.4. Shared complete-key filtering for one scan group. A candidate's full primary score is computed before admission. A valid c-candidate witness set can tighten the per-query bound; a stale but complete bound only weakens filtering. Final prefix trimming reduces the valid merge input, while the allocated workspace capacity remains unchanged. The whole-key-read premise is qualified in the text.

rule is

$$g_{\text{fill}} = \max(\{1\} \cup \{h \in \{2,4,8\} : bhT_{\text{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\text{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\text{chosen}} = \max(g_{\text{fill}}, g_{\text{skew}}). \tag{5.18}$$

The skew rule may override the fill limit. In the evaluation harness, v is calibrated using 512 database vectors as pseudoqueries and a fixed routing configuration; it uses no ground-truth labels. This is distinct from Chapter 6's sampled output-agreement calibration. Once calibrated, scheduling uses the batch size and stored statistic without an additional per-search host readback. The constants were fitted on four pilot configurations and then evaluated on further configurations. The one-time pseudoquery calibration is excluded from the reported steady-state search times. Observed startup ranges, separate from refinement-budget calibration (section A.4), are

$$T_{\text{gather}} \in [0.81, 7.04]\,\text{ms}, \quad T_{\text{first}} \in [6.00, 27.77]\,\text{ms}, \quad \text{median}(T_{\text{repeat}}) \in [2.72, 18.13]\,\text{ms}.$$

Choosing one group dispatches $g_{\text{scan}} = 0$ at fixed index/probes/budget. Its distance-only ties need a common-order check for identical survivors. Integrated gains/regret are in section 7.6.

5.5.4 Observed execution paths

On the 170-SM RTX 5090 (48 warps/SM) [24],

$$1000 \cdot 2 \cdot 512 = 1{,}024{,}000 > 0.6 \cdot 170 \cdot 48 \cdot 32 = 156{,}672.$$

Thus batch 1,000 selects one group by fill; skew may force four.

Nine of 38 E1 records cross $v = 0.3$, all Vogue (0.338–0.622). Across 132 run–budget cells, Vogue uses four groups in 27; arXiv/OpenAI use $g_{\text{scan}} = 0$ in 105. Their full-batch gains use LUTs, packed access, lane refinement and probe selection, without grouped retention/shared bounds.

Vogue has 3–15 tie-only primary differences per 1,000 queries, zero non-tie differences and bit-identical final outputs. The 7–11 range is specific to 128 probes.

Observed cell counts are

	$G = 0$	$G = 2$	$G = 4$
held-out	10	0	2
$S = 128$	2	8	2

Eight of twelve switch paths. Grouping serves smaller/skewed loads, without a fixed batch-256 boundary.

The heuristic omits register/shared residency and compaction/merge costs; pseudoqueries may age. Worst recorded regret is 1.217 at batch 256. These tests hold the budget fixed; section 5.6 refines survivors.

5.6 Lane-Cooperative Residual Scoring and Final Top-k

Primary screening limits residual-coordinate work to $O(cd)$ instead of $O(n_c d)$. Lanes cooperate at the same budget.

5.6.1 Reconstructed scoring and the query-only constant

With $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$, real-arithmetic expansion gives

$$D_{\mathrm{H}}(q, x) = D_{\mathrm{P}}(q, x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) \tag{5.19}$$

$$= 2\|q'\|_2^2 + \|\widehat{p} + \widehat{r}\|_2^2 - 2\langle q', \widehat{p} + \widehat{r} \rangle$$

$$= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. \tag{5.20}$$

The query-only constant preserves ranking; reconstruction error remains [16].

The scalar path sums squared coordinate differences between q' and residual reconstruction, then adds the primary score/correction. Queries and fitting codebooks use shared memory.

5.6.2 Mapping lanes to words and reducing partial sums

For $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$,

$$d_{\mathrm{word}} = 32/B_R, \qquad \text{words of lane } a: \ a, a + L, a + 2L, \ldots.$$

A word holds four eight-bit or eight four-bit codes. Butterfly shuffles sum lane partials; lane zero writes the score.

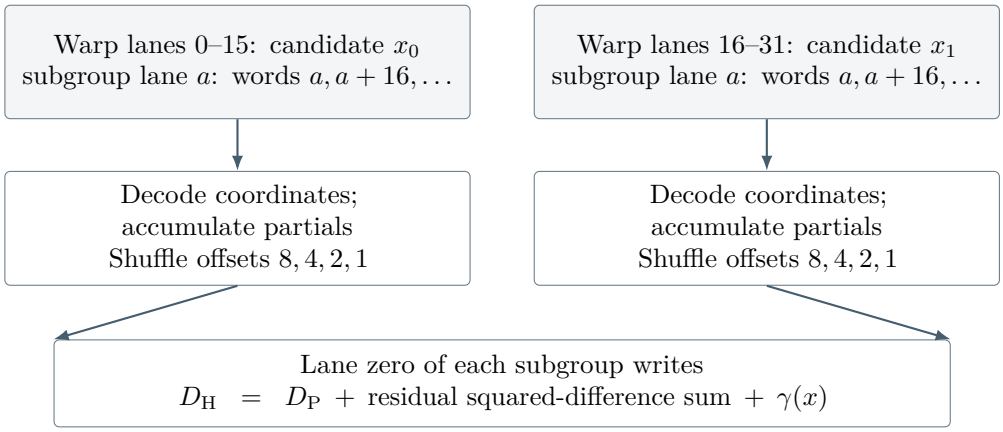


Figure 5.5. Sixteen-lane residual refinement. A warp handles two candidates; lane a within a candidate reads residual words $a, a + 16, \ldots$. Each eight-bit-code word decodes four coordinates. Subgroups sum partials, then add primary score/correction. Candidate positions may remain scattered.

Require word-aligned rows and $d_{\text{word}} \mid D_s$. Every lane joins shuffles; invalid slots contribute zero before receiving infinite output scores. $L = 16$ assigns two candidates per warp.

L0 loads scalar bytes; L1 loads words with the same one-lane accumulation. L8/L16/L32 trade shorter walks for shuffles and fewer concurrent candidates. FP32 association, locality and codebook placement affect the result.

5.6.3 Final selection and the numerical boundary

For at least k finite valid survivors with unique IDs, each round computes

$$i^* = \arg\min_{i<c}(D_{H,i}, \mathrm{ID}_i), \quad d^* = D_{H,i^*}, \quad D_{H,i^*} \leftarrow +\infty$$

using all slots and emits (ID_{i^*}, d^*): exact retained-pool top-k, without thread quotas. After finite entries run out, a smallest valid ID may repeat at infinity; all-invalid pools return invalid IDs. Evaluated pools meet the finite-entry contract.

Final ties use database IDs; primary ties use positions. Refinement cannot recover discarded candidates or quantisation loss. Lane FP32 association can move near ties.

5.7 IVF Probe Selection as an Independent Primitive

Probe selection precedes primary scanning and selects inverted lists, not database-vector survivors. For IVF centroid μ_i, the selector orders the FP32-valued computed routing scores

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

Omitting the common query norm preserves centroid ordering. In contrast to primary squared-distance tables, these scores can be negative. The required output is the first $n_{\text{sel}} = \min(n_{\text{probe}}, n_{\text{list}})$ centroid IDs in score order, using the smaller list ID at a tie, plus the prefix offsets of their list lengths.

OpenAI rotation and centroid GEMMs use TF32 inputs with FP32 accumulation in v69. Probe selection is exact on these computed scores; later FP32 `fma` retains GEMM rounding (section 7.2.3).

5.7.1 Radix selection followed by deterministic ordering

One block makes four radix passes, most-significant byte first. Each histogram counts the chosen prefix; its selected bin updates the residual rank. Four bytes determine the cutoff and required equal-key count.

Canonicalise signed zeros to positive zero, then map bits u by

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

For non-NaN scores, monotonicity follows by cases:

$$\begin{aligned} &D_1 < D_2 < 0 \Rightarrow u_1 > u_2 \Rightarrow \sim u_1 < \sim u_2, \\ &0 \le D_1 < D_2 \Rightarrow u_1 < u_2 \Rightarrow (u_1 \mid 2^{31}) < (u_2 \mid 2^{31}), \\ &D_1 < 0 \le D_2 \Rightarrow F(u_1) < 2^{31} \le F(u_2). \end{aligned}$$

This covers infinities and equal zeros, unlike equation (5.16)'s nonnegative-only domain. Every pass uses the same rounded `fma`, fixing recomputed keys.

Histogram storage is

$$256 \text{ bins} \times 32 \text{ lane replicas} \times 4 \text{ bytes} = 32\,\text{KiB}.$$

Replicas separate same-warp updates, with inter-warp contention and reduction overhead remaining.

Let h be the cutoff key. Collection forms

$$A_< = \{i : F(u_i) < h\}, \quad A_= = \{i : F(u_i) = h\}, \quad m = n_{\text{sel}} - |A_<|.$$

It takes $A_<$ and the m smallest list IDs from $A_=$. Ballots/prefix counts enforce this order, independent of atomic arrival. For the resulting set A,

$$\text{rank}(i) = |\{j \in A : (s_j, j) \prec (s_i, i)\}|, \qquad \text{out}[\text{rank}(i)] = i.$$

No full centroid sort is needed.

Selected lengths produce prefix offsets. For excess probes,

$$i_j = i_{n_{\text{sel}}-1}, \quad L_j = 0 \quad (j \geq n_{\text{sel}}), \qquad \omega_{j+1} = \omega_j + L_j.$$

Repeated IDs add no scan work. Centroid sets and probe counts must be positive.

5.7.2 Costs and separate correctness contracts

Histogram/collection reread scores; ranking costs $O(n_{\text{sel}}^2)$ with growing shared storage. Small selections still pay fixed histogram allocation. The tests assess deterministic integration of this radix method.

Section 7.4 tests ties, signed zeros, infinities, extremes and excess probes.

FP16 routing is a separate approximation in section 7.4, alongside cross-process identity checks.

6 Workload-Level Budget Calibration

Unlabelled agreement with a larger-budget reference selects one multiplier for later batches. Quality and calibration cost determine whether reuse helps.

6.1 Why Fixed Refinement Budgets Can Be Wasteful

For routed $U(q)$, the refinement multiplier sets [16]

$$c(\alpha) = \max\{k, \lceil \alpha k \rceil\}, \qquad C_\alpha(q) = \mathrm{Top}^{\mathrm{primary}}_{\min\{c(\alpha), |U(q)|\}} U(q). \tag{6.1}$$

Require $|C_\alpha| \geq k$ for the analysis. Underfull slots are sentinels. Budgets change retention; probes change U.

The $k = 10$ grid retains 20–2,000 survivors without retraining, trading residual reads against missed neighbours. Stored residuals and previously grown workspace remain allocated.

At fixed batch/routing, write

$$T(\alpha) = T_{\mathrm{base}} + T_{\mathrm{select}}(c(\alpha)) + T_{\mathrm{refine}}(c(\alpha)) + T_{\mathrm{final}}(c(\alpha)). \tag{6.2}$$

T_{base} covers transfers, rotation, routing and primary arithmetic; budget-dependent selection/compaction follows figure 6.1. Capacity/resource thresholds prevent a monotone-latency claim. The rule minimises accepted budget, not noisy QPS (section 7.5.5).

Every trial reruns rotation, probes and primary scoring. Index reuse avoids training/building, rather than caching query intermediates. Calibration therefore counts complete searches.

One budget serves the index, route, k and workload. It is not a per-query predictor. Scan scheduling remains separate (section 5.5) and can change the calibration path.

Position among adaptive ANN methods. α serves the candidate-expansion role of Faiss's `k_factor` and cuVS refinement [7, 10]. Alternatives use GT tuning [9], learned per-query stopping [19] or predicted recall [6]. Here unlabelled agreement chooses one reusable workload budget. Scores still use reconstructed vectors, unlike the cuVS source-vector refinement example.

6.2 GT-Free Sampled Output Agreement

Let $\mathcal{R}_\alpha(q)$ be the set of k valid, unique database IDs returned at budget α on a fixed index. Define $\mathcal{R}_{\mathrm{ref}}(q) = \mathcal{R}_{\alpha_{\mathrm{ref}}}(q)$. The descending grid is

$$A = (200, 100, 64, 32, 16, 8, 4, 2), \qquad \alpha_{\mathrm{ref}} = A[0] = 200. \tag{6.3}$$

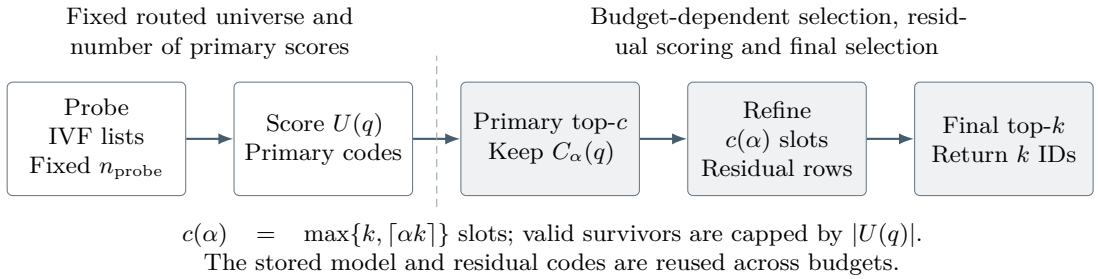


Figure 6.1. The refinement budget changes work after primary-score production. With a fixed query, index and probe count, it leaves the routed candidate universe unchanged. The highlighted stages depend on survivor capacity; this is a logical work decomposition, not a measured latency breakdown. The scan kernel also performs budget-dependent admission and compaction. Padding applies when fewer than c candidates exist.

For S calibration queries $\mathcal{Q}_S$, define

$$\begin{aligned} D_S(\alpha) &= \sum_{q \in \mathcal{Q}_S} (k - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{\text{ref}}(q)|) \\ &= \frac{1}{2} \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_\alpha(q) \triangle \mathcal{R}_{\text{ref}}(q)|. \end{aligned} \tag{6.4}$$

Accept iff $D_S(\alpha) \leq \epsilon = 1$: at most one missing reference ID, ignoring order/distances. The finite-grid objective is

$$\widehat{\alpha} = \min\{\alpha \in A : D_S(\alpha) \leq \epsilon\}. \tag{6.5}$$

The reference is a fallback. Section 6.3 gives the conditions for exact bisection.

At $k = 10$, $\epsilon = 1$ gives

S	32	64	128	500
$\epsilon/(Sk)$	0.003125	0.0015625	0.00078125	0.0002

Larger samples change coverage and strictness. Disjoint held-out queries evaluate draws from one finite pool (section 7.5).

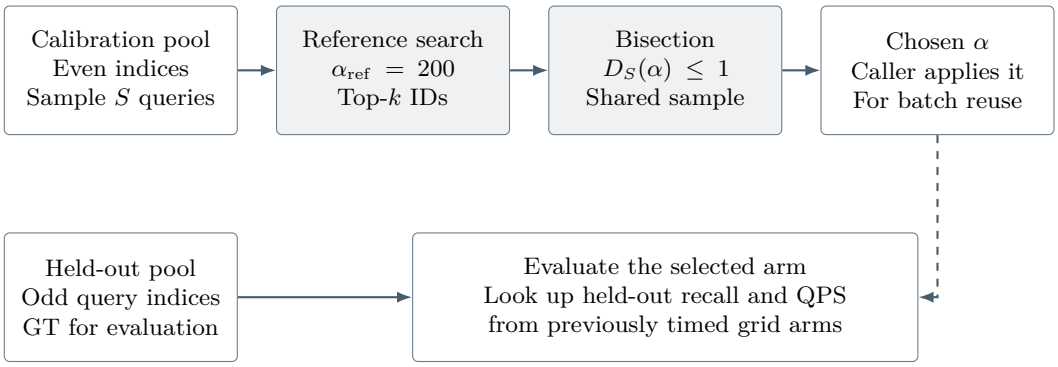


Figure 6.2. Workload-level calibration and its evaluation. Budget decisions use reference IDs; GT assesses held-out quality. Selected-arm timing comes from fixed-arm measurements. Implementation boundaries are in section A.3.

6.2.1 What agreement controls

Under Proposition 6.1, disagreement counts reference answers excluded by primary screening:

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_{\text{ref}}(q) \setminus C_\alpha(q)|. \tag{6.6}$$

For k-element outputs and GT set $\mathcal{T}_k$, set difference gives

$$|\mathcal{R}_{\text{ref}} \cap \mathcal{T}_k| - |\mathcal{R}_\alpha \cap \mathcal{T}_k| \leq |\mathcal{R}_{\text{ref}} \setminus \mathcal{R}_\alpha|.$$

Summing and dividing by Sk yields the relative bound. On the same sample,

$$\begin{aligned} r_{\mathcal{Q}_S}(\alpha) &\geq r_{\mathcal{Q}_S}(\text{ref}) - \frac{D_S(\alpha)}{Sk}, \\ D_S(\alpha) \leq \epsilon \implies r_{\mathcal{Q}_S}(\alpha) &\geq r_{\mathcal{Q}_S}(\text{ref}) - \frac{\epsilon}{Sk}. \end{aligned} \tag{6.7}$$

Scope. The bound is relative to the reference on this sample. GT evaluation measures the reference's remaining error (section 7.8.2).

6.2.2 Sampling resolution and ordered testing

Let H_S count queries with any missing reference ID. Since $H_S \le D_S$, acceptance implies $H_S \le 1$. For a prespecified budget and i.i.d. queries, the one-event one-sided 95% binomial limit solves [22]

$$(1-u)^S + Su(1-u)^{S-1} = 0.05. \tag{6.8}$$

The model-dependent resolutions are

	$S = 128$	$S = 500$
i.i.d. query events	0.03652	0.009452
i.i.d. homogeneous result slots	0.003701	0.0009484

The stronger slot model gives approximately $4.74/(Sk)$. Query i.i.d. alone cannot justify this division by k; neither model certifies fixed-pool subsampling.

Monotone disagreement gives a prefix of accepted indices, allowing fixed-sequence control with valid per-arm tests [3]. Here the one-slot threshold measures stability, not certified GT risk.

A finite-pool extension. Assume each query has fixed $X_q(\alpha)$ at the intended execution regime, independent of subset membership/position. For pool size N_{cal} and counts $n_j = |\{q : X_q = j\}|$, uniform sampling without replacement accepts only all-zero samples or one single-count query [23]:

$$\Pr\{D_S(\alpha) \le 1\} = \frac{\binom{n_0}{S} + n_1\binom{n_0}{S-1}}{\binom{N_{\text{cal}}}{S}}. \tag{6.9}$$

Out-of-range binomial coefficients are zero. Predicting GT risk also needs the grid acceptance structure and held-out fixed-arm quality. Subset sums check invariance empirically (section 7.5.5).

6.3 Budget Search, Nestedness and Tie Safety

6.3.1 Bisection and complete enumeration

Write $J = |A|$. The implementation computes the sample reference once, then searches for the largest acceptable index in the descending grid:

1. Set $l = 0$, $h = J - 1$. Index zero is the reference fallback.
2. While $l < h$, test $j = \lfloor (l + h + 1)/2 \rfloor$ by running $A[j]$ and computing $D_S(A[j])$.
3. If $D_S(A[j]) \le \epsilon$, set $l = j$; otherwise set $h = j - 1$.
4. Return $A[l]$. A caller applies it before processing reuse batches.

The eight-point grid needs at most three complete candidate searches in addition to the reference. Invocation, host-comparison complexity and timing boundaries are specified in section A.3.

Enumeration reruns the reference and all seven budgets on the same sample. Both timers include a fresh reference. Enumeration handles acceptance holes; bisection matches its minimum only when accepted indices form a prefix.

6.3.2 A sufficient condition for monotonicity

Proposition 6.1 (Nested survivors preserve reference membership)**.**
For one query, assume fixed routing and survivor sets $C_\alpha \subseteq C_\beta \subseteq C_{\alpha_{\text{ref}}}$ whenever $\alpha \leq \beta \leq \alpha_{\text{ref}}$. Assume at least k valid survivors, fixed refined scores and one deterministic total order for final ties. Each result is the exact top-k in that order. Then

$$|\mathcal{R}_\alpha \cap \mathcal{R}_{\text{ref}}| = |C_\alpha \cap \mathcal{R}_{\text{ref}}|, \qquad D_S(\alpha) \geq D_S(\beta). \tag{6.10}$$

The latter statement holds when these conditions hold for every sampled query.

Proof.
For $x \in \mathcal{R}_{\text{ref}} \cap C_\alpha$, the common total order gives

$$|\{y \in C_\alpha : y \prec x\}| \leq |\{y \in C_{\alpha_{\text{ref}}} : y \prec x\}| < k.$$

Hence $x \in \mathcal{R}_\alpha$. Together with $\mathcal{R}_\alpha \subseteq C_\alpha$, this proves the intersection identity in equation (6.10). Nestedness gives $|C_\alpha \cap \mathcal{R}_{\text{ref}}| \leq |C_\beta \cap \mathcal{R}_{\text{ref}}|$. Summing missing reference counts over queries proves $D_S(\alpha) \geq D_S(\beta)$. □

Fixed-key exact primary top-c gives nesting; fixed-key refinement then gives monotone agreement. This supports bisection, without making GT recall or runtime monotone as the pool grows.

6.3.3 A corollary for the evaluated grouped path

Within a draw, S, routing and scan threads are fixed; `scheduled_groups(S)` ignores c. Grouped budgets use the same LUT sums. Fixed B_r, 16 lanes and final $(D_{\text{H}}, \text{ID})$ order keep candidate scores fixed.

Corollary 6.2 (Grouped calibration)**.**
For the evaluated grouped configuration, under the representation and shared-bound contracts of Chapters 4 and 5, primary survivors are nested across budgets. Consequently Proposition 6.1 applies and bisection returns the same minimum accepted grid budget as enumeration.

Proof.
Lemma 5.1 and propositions 5.1 and 5.2 give exact primary top-c under the common key $(D_{\text{P}}, \text{position})$. Its prefixes nest as c grows. The fixed per-candidate refinement and exact final total order supply the remaining premises of Proposition 6.1. □

The corollary covers 2,176/2,304 draws; 128 OpenAI-3072 draws at $S = 128$ use distance-only selection. Every recorded curve is monotone and matches enumeration, a finite check (section 7.5.5).

Full-pool coverage. Two FULL cells group Vogue; ten use $g_{\text{scan}} = 0$. All twelve match enumeration. The ten distance-only cells lie outside corollary 6.2; general nesting needs a common cutoff order or no tie crossing grid/reference cutoffs. Sharing the held-out path supplies neither condition.

Case	C_1	C_2	C_3	Reference overlap
Nested	$\{b\}$	$\{b, c\}$	$\{a, b, c\}$	0, 0, 1
Non-nested	$\{a\}$	$\{b, c\}$	$\{a, b, c\}$	1, 0, 1

Figure 6.3. Constructed $k = 1$ example with refined order $a \prec b \prec c$ and reference $\{a\}$. The nested case preserves reference membership. With all primary scores tied, different tie choices can produce the non-nested middle pool and break monotonicity. These are explanatory sets, not measured GPU outputs.

A rank-based extension. Under the same nesting/fixed-score premises, one-based primary ranks give

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \sum_{x \in \mathcal{R}_{\mathrm{ref}}(q)} \mathbf{1}\{\mathrm{rank}_{\mathrm{P}}(x) > c(\alpha)\}. \tag{6.11}$$

A reference-rank export could recover all grid counts from one search. The harness instead reruns budgets; rank extraction remains future work.

6.4 Budget Baselines and Their Questions

Table 6.1 separates budget selection from its evaluation. Fixed arms share the index, route and held-out queries; invocation and timing details are in section A.3.

For held-out recall/QPS $\widehat{r}, \widehat{Q}$ and $\widehat{r}^* = \max_{\alpha \in A} \widehat{r}(\alpha)$,

$$\alpha_{\mathrm{oracle}} \in \arg\max_{\alpha \in A} \left\{ \widehat{Q}(\alpha) : \widehat{r}(\alpha) \geq \widehat{r}^* - \delta \right\}, \qquad \delta = 10^{-3}. \tag{6.12}$$

This diagnostic uses labels/timings; its recall slack δ differs from the slot allowance ϵ.

FULL uses label-free bisection with tighter $\epsilon/(Sk)$; same-sample enumeration checks its search. Agreement minimises accepted budget, while the oracle maximises quality-constrained QPS.

Fixed 64/200 test smaller/high budgets (section 7.5 and table 7.8).

6.5 Hold-out Risk, Calibration Cost and Payback

6.5.1 Choosing a calibration scale

The strong independent-slot model suggests

$$\frac{4.74}{Sk} \simeq 10^{-3}, \quad k = 10 \quad \Longrightarrow \quad S \simeq 474 \approx 500.$$

This resolves reference disagreement, not GT risk (section 6.2). FULL quality and sampled risks are in table 7.7 and figure 7.6.

FULL still needs the path/order conditions in section 6.3.

6.5.2 Risk and selected-arm performance

Over m calibration draws, let draw r select $\widehat{\alpha}_r$. Define held-out loss and threshold-crossing frequency by

$$L_r = \widehat{r}^* - \widehat{r}(\widehat{\alpha}_r), \qquad \widehat{p}_\delta = \frac{1}{m} \sum_{r=1}^{m} \mathbf{1}\{L_r > \delta\}. \tag{6.13}$$

Table 6.1. Budget controls and the questions they answer. Held-out GT is used to assess every result, but only the diagnostic oracle uses it to choose a budget. The reference is fixed at 200 for agreement-based arms.

Control	Selection information	Question
Fixed 100	No calibration	Gain relative to the inherited default?
Fixed 64 and smaller grid arms	No calibration	Does a simple fixed choice remove the need for calibration?
Sampled bisection	S unlabelled queries; slot agreement	How well does a low-cost sample choose a reusable budget?
Same-sample enumeration	Same S, reference and threshold; all grid arms	Did bisection miss the smallest accepted grid budget?
Full-pool calibration	All 500 calibration queries; same bisection rule	What changes with broader coverage and a stricter normalised allowance?
Held-out oracle	GT recall and timed QPS for all arms	Which measured arm is fastest under the diagnostic quality constraint?

Mean loss, crossing frequency and tail size answer different questions. Table 7.8 and figure 7.6 report the archived summaries; the largest supplementary per-draw losses are reported in section 7.5.5.

Reported selected-arm rate is $\overline{Q} = m^{-1} \sum_r \widehat{Q}(\widehat{\alpha}_r)$, a lookup from timed fixed arms. It is distinct from the median budget's rate and from the throughput of a calibrated query stream. Quality and speed must be read together (table 7.9).

6.5.3 Cost of reuse

For B_{reuse} equal-sized subsequent batches, model

$$T_{\text{policy}}(B_{\text{reuse}}) = T_{\text{cal}} + B_{\text{reuse}} T_{\text{chosen}}, \qquad T_{\text{fixed}}(B_{\text{reuse}}) = B_{\text{reuse}} T_f. \tag{6.14}$$

When $T_{\text{chosen}} < T_f$, the break-even count is

$$B^*_{\text{reuse}} = \frac{T_{\text{cal}}}{T_f - T_{\text{chosen}}}. \tag{6.15}$$

Indeed,

$$\begin{aligned} T_{\text{policy}} < T_{\text{fixed}} &\iff T_{\text{cal}} < B_{\text{reuse}}(T_f - T_{\text{chosen}}) \\ &\iff B_{\text{reuse}} > B^*_{\text{reuse}} \qquad (T_{\text{chosen}} < T_f). \end{aligned}$$

If $T_{\text{chosen}} \geq T_f$, there is no throughput payback. Batch time is $T = 1000n/Q$ milliseconds. For varying choices, convexity gives $1/\mathbb{E}[Q] \leq \mathbb{E}[1/Q]$: inverse mean QPS understates mean time. Sections A.3 and 7.5.6 give boundaries and four direct cost/quality checks. Campaign estimates are in table 7.9.

Recalibrating every H batches costs T_{cal}/H per batch. Drift detection/automatic renewal remain unimplemented.

6.6 Policy Failure Modes and Applicability

Reuse needs a stable index, k, routing and workload. Ordering/path conditions are in section 6.3; quality/tails/cost are in section 6.5. Drift detection remains future work.

The caller must explicitly install α before reuse.

The budget campaign and trace rerun use v69, including OpenAI TF32. FP32 frontier checks neither validate nor recalibrate those choices.

7 Experimental Evaluation

We test construction/capacity (RQ1), execution (RQ2), budget quality/reuse (RQ3) and competitive regions (RQ4).

7.1 Experimental Protocol

7.1.1 Comparison protocol and reproducibility

Sections 3.2, 3.3 and 3.5 define the protocol. Search excludes construction/calibration; each ablation has its own baseline.

CPU JHQ uses unmatched settings/protocols: October Vogue has 1,024 lists and $[8, 4]$ bits versus E1's 4,096 lists. A matched CPU/GPU speedup is unavailable.

Adaptive calls/device-only timing are in section A.2.

Metrics and frontier estimators follow sections 3.2 and 3.4; g_{LUT} and g_{scan} retain the distinct meanings in section 5.1.

7.2 Modern GPU ANN Recall–QPS Frontiers

7.2.1 Comparison scope and frontier construction

Figure 7.1 compares JHQ-GPU with the graph, compressed-IVF and brute-force baselines of section 3.4 [17, 28, 33], using the first four datasets in table 3.2. Each uses one batch of 1,000 queries with common GT. T5 separately evaluates small batches in section 7.2.5; section 7.6 reports internal ablations.

The recorded schedule uses $g_{\mathrm{scan}} = 0$ for all 105 arXiv/OpenAI run–budget cells and $g_{\mathrm{scan}} = 4$ for all 27 Vogue cells, triggered by Vogue's candidate-count CV of 0.338–0.622. Thus the non-Vogue frontiers exercise query-centric scanning, while Vogue also exercises grouped retention, shared bounds and merging (section 5.5).

Applying the frontier rule in section 3.4, JHQ contributes 27–39 points per dataset and each baseline family 54–264. E1 contains 1,590 valid points, no failure records and no invalid-ID or duplicate-row flags. These points share indexes across parameter sweeps.

Grids are in table 3.3 and section B.1. A curve ending short of a target has no tested coverage there.

Nine-bit RaBitQ shares the nominal $9d/8$ payload; 4/8-bit codes use less, so storage is unmatched. Table 4.1 and section 7.7 give representation costs and measured allocations alongside this performance envelope.

7.2.2 Matched recall and opposing operating regions

Table 7.1 reports recall 0.97 using the interpolated and qualifying-measured estimators defined in section 3.4.

At batch 1,000/recall 0.97, removing grouping drops Vogue's JHQ/RaBitQ ratio from 1.059 to 0.933 at fixed index, probes, budget and final outputs. This is an integrated grouping benefit; the bound alone is not isolated.

Table 7.1. Recall 0.97 comparison. Each ratio is interpolated / measured; values above one favour JHQ. JHQ kQPS is the interpolated estimate. The small arXiv–RaBitQ difference changes direction with the estimator.

Dataset	JHQ kQPS	vs CAGRA fp32	vs IVF-RaBitQ
Vogue	230.8	1.416 / 1.537	1.059 / 1.046
arXiv	258.6	1.317 / 1.422	0.996 / 1.015
OpenAI-1536	126.7	1.747 / 2.294	1.130 / 1.111
OpenAI-3072	55.9	1.747 / 2.088	1.157 / 1.346

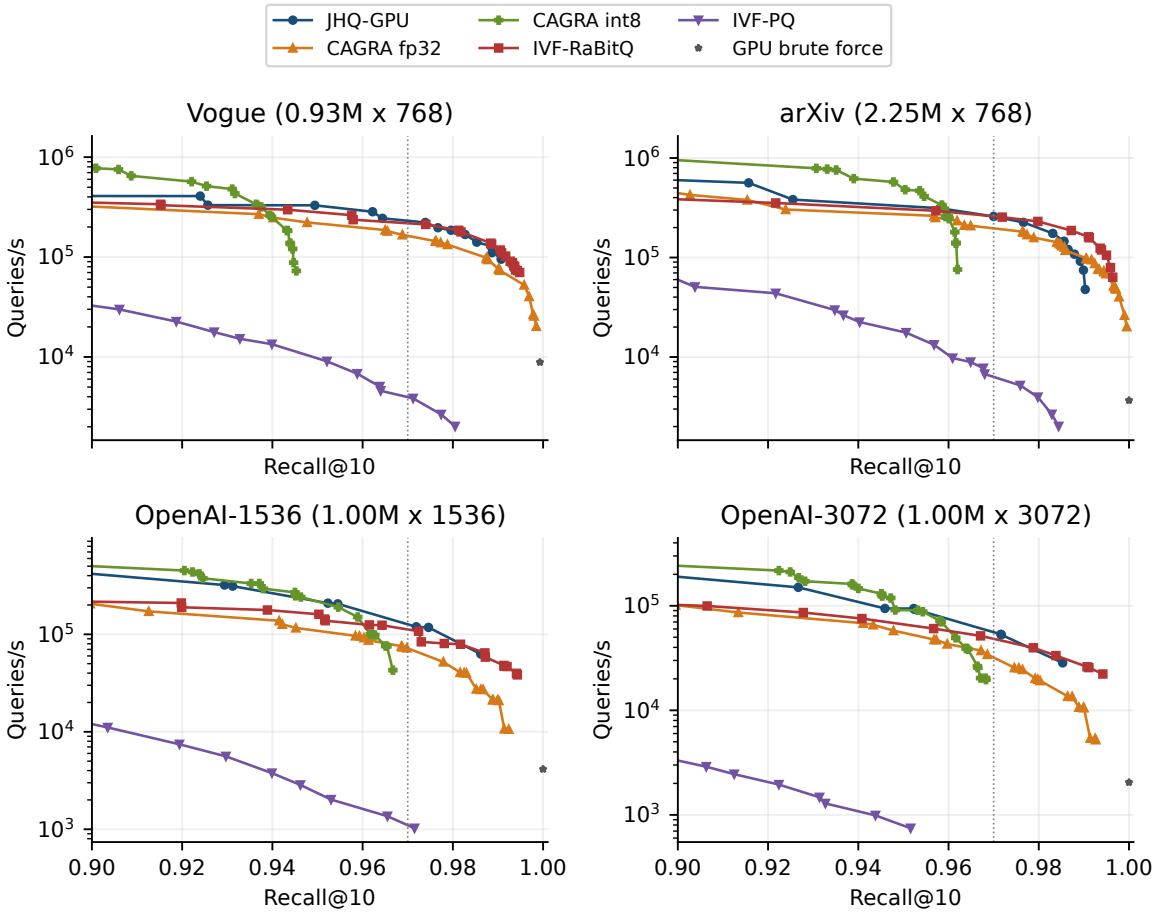


Figure 7.1. Measured host-to-host recall–QPS frontiers on the four E1 datasets. The displayed recall window begins at 0.90, with a vertical line at 0.97; all baseline families, including CAGRA int8, participate in frontier construction. Lines connect measured points; throughput uses a logarithmic axis.

CAGRA int8 leads by 1.20–1.98× at recall 0.90–0.92 but lacks 0.97 coverage. RaBitQ wins some 0.98+ targets; CAGRA fp32 wins near 0.99 on arXiv (figure 7.1).

7.2.3 Precision sensitivity on the OpenAI datasets

Supplementary source inspection establishes that the E1 implementation uses TF32 multiplication with FP32 accumulation for OpenAI rotation and centroid GEMMs, despite FP32 input/storage and a misleading source comment (section 4.2). The main E1 frontier remains the historical measurement of that implementation. In campaign D, a separate FP32 variant changes the cuBLAS math-mode switch and replays 37/39 OpenAI configurations at batch 1,000, five rounds each. Execution order alternates between variants across probe settings. Both reuse the same TF32-trained model; the intervention changes add/query arithmetic, including database encoding and routing. It is not a comparison of independently FP32-trained models. Table 7.2 summarises the paired changes.

Table 7.2. FP32 sensitivity. QPS ratio is FP32/TF32 at equal configuration (geometric mean); recall differences have the same direction. The last two columns compare the FP32 frontier at recall 0.97 with archived E1 IVF-RaBitQ and CAGRA fp32 frontiers. Campaign D does not remeasure the baselines.

Dataset	Pairs	QPS ratio	ΔRecall	vs RaBitQ	vs CAGRA
OpenAI-1536	37	0.953	+0.0003–+0.0005	1.086	1.679
OpenAI-3072	39	0.951	+0.0000–+0.0003	1.079	1.629

At equal configuration, FP32 reduces throughput by about 5% and increases recall by 0–0.0005. At recall 0.97, FP32 is about 3% slower than same-session TF32, yet 1.086/1.079× archived RaBitQ and 1.679/1.629× archived CAGRA fp32. New-session v69 gives 1.118/1.114 against archived RaBitQ, versus historical 1.130/1.157. D compares baselines across sessions; only v69/v72 share a session. Figure 7.2 shows the two JHQ frontiers with their archived comparators. T1 separately remeasures same-session anchors, with grid-sensitive results in section B.4.1.

7.2.4 Interpretation and comparison limits

Both OpenAI datasets use Qdrant DBpedia embeddings [31, 32]. After truncation/renormalisation, 1,000 query and 10,299 sampled base rows have cosine $\geq$ 0.9980 (median 1.000000), with GT overlap 9.046/10. This supports row alignment without entity IDs. Doubling width leaves the CAGRA ratio at 1.747 and changes RaBitQ's 1.130 to 1.157: width alone does not explain the gain.

Gains accompany smaller 128-probe anchors: OpenAI 4–8, Vogue 64, arXiv 100 (later FULL 200). This is an association, not an isolated effect or matched-frontier setting.

Section 7.8 checks sensitivity to small margins and the interface/memory/compilation differences of section 3.3.

7.2.5 Small batches change the competitive region

T5 compares scheduled JHQ, CAGRA fp32/int8 and IVF-RaBitQ at batches 8/32 on the four E1 datasets. Five synchronous host-to-host rounds use 1,000/992 queries, respectively. Its 1,808 points include JHQ's one-block control; IVF-PQ and brute force are outside this campaign. Figure 7.3 and table 7.3 compare the resulting regions.

Table 7.3. T5 QPS ratios: interpolation with qualifying endpoints / strict measured threshold. The fastest competitor is selected independently for each estimator; higher-recall qualifying endpoints remain eligible without extrapolation.

Dataset	b	$R = 0.90$	$R = 0.95$	$R = 0.97$
Vogue	8	1.155 / 1.091	1.073 / 0.980	0.893 / 0.854
Vogue	32	1.069 / 0.997	0.956 / 0.895	0.865 / 0.839
arXiv	8	1.479 / 1.454	0.859 / 0.851	0.697 / 0.698
arXiv	32	1.166 / 1.148	0.667 / 0.661	0.622 / 0.582
OpenAI-1536	8	1.316 / 1.131	0.851 / 0.825	0.641 / 0.620
OpenAI-1536	32	1.071 / 0.915	0.770 / 0.752	0.937 / 0.914
OpenAI-3072	8	1.032 / 0.953	0.798 / 0.798	0.721 / 0.702
OpenAI-3072	32	0.866 / 0.734	0.608 / 0.771	1.008 / 1.351

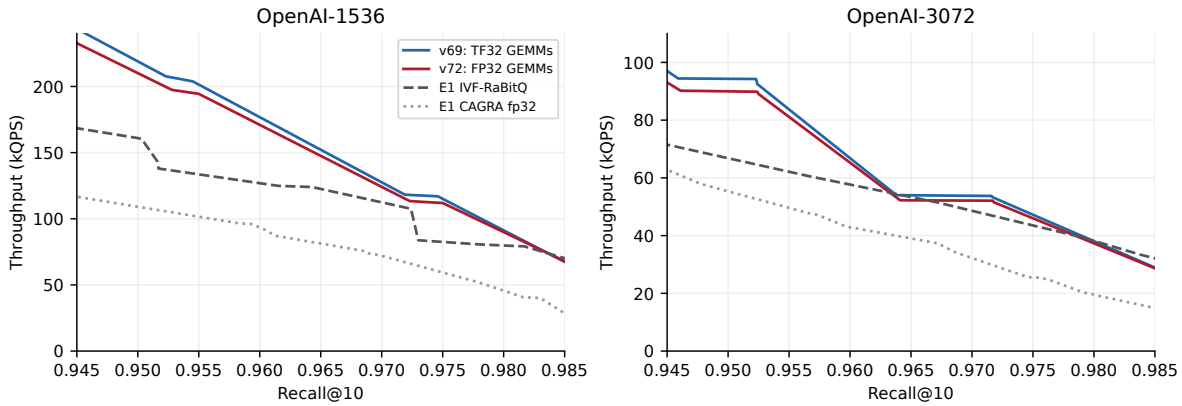


Figure 7.2. Campaign D precision frontiers. Solid curves are same-session JHQ variants; dashed/dotted baselines are historical E1 measurements. All use the chapter's frontier and interpolation rules.

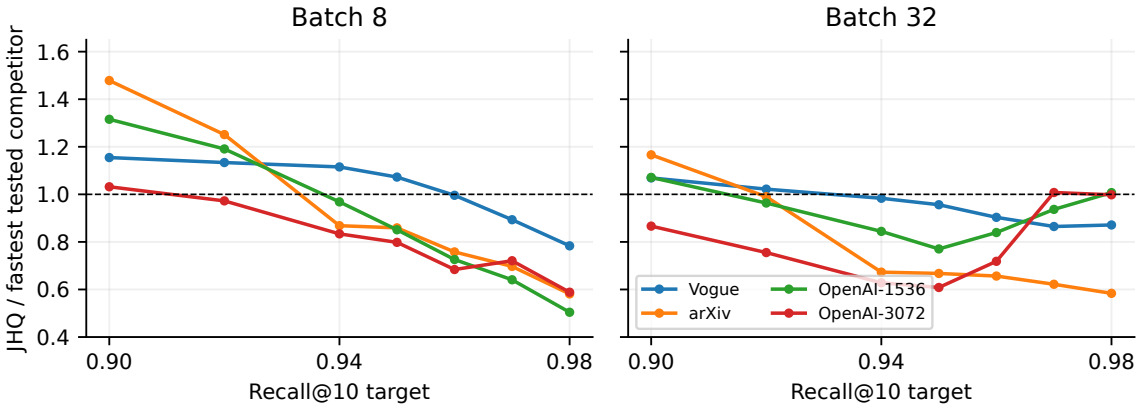


Figure 7.3. T5 JHQ / fastest-competitor QPS under interpolation with qualifying measured endpoints. Above one favours JHQ. Table 7.3 gives strict thresholds.

At recall 0.97 and batch 8, CAGRA fp32 leads on all four datasets under both estimators: interpolated JHQ ratios are 0.641–0.893. Batch-32 ratios are 0.622–1.008: OpenAI-3072 is near parity under interpolation, JHQ-favourable under the measured threshold. Vogue's batch-8/0.95 lead also depends on interpolation. Borderline rankings depend on grid resolution.

Qualifying endpoints above the target remain eligible. At recall 0.90, batch-8/32 ratios are 1.032–1.479/0.866–1.166, with three/one strict-threshold JHQ wins. Dropping endpoints overstates leads. Scheduling gains 1.7–3.3× over one block at 0.95, without ensuring cross-system wins.

Null-call medians are 0.012–0.072 ms and remain included. CAGRA uses Python, JHQ/RaBitQ C++; warm-ups differ (section A.5).

7.3 Cartesian LUT, Packed Loads and Lane Refinement

The RQ2 ablations measure both kernel and application effects.

7.3.1 Exact table factorization and its throughput consequence

The LUT ablation tests the cost model in section 5.2 and FP32 ranking effects.

Figure 7.4 compares Cartesian table grouping on six datasets, probe counts 8/32/128/512, fixed $\alpha = 100$, and $g_{\text{LUT}} \in \{1, 2, 4, 8\}$. Five rounds use batch 1,000 (Stella 999). Two groups lead in 22/24 cells. Exceptions are near ties: arXiv/128 favours one group by 0.23%; OpenAI-3072/8 favours four by 0.84%. Paired two-versus-one wins are 23/24; all dataset geometric means exceed one (1.021–1.318). Shared-index cells are not independent trials.

The two-group/one-group ratio ranges from 0.998 to 1.914. Four and eight groups each store 16 entries but require more lookups and additions. These arms show that smaller tables need not improve throughput; occupancy and instruction costs were not isolated. The maximum reported recall spread within any four-arm cell is 10^{-4}. At 128 probes and $\alpha = 100$, T4 compares identical candidates on Vogue, arXiv and OpenAI-3072. Full versus $16 + 16$ tables differ in 22.6–26.1% of candidate-score bit patterns, with maximum relative error 4.53×10^{-7}. Across all forms, 20.5–29.4% differ, with maximum relative error 4.64×10^{-7}. Vogue has 2–4/256 final-ID differences, while arXiv/OpenAI have none. Unchanged primary survivor sets can still produce different final IDs because the rounded primary score enters refinement (section A.5).

7.3.2 Packed access and cooperative residual scoring

Packing wins 1.012–1.658× across 48 matched pairs (six datasets, four probe counts, $g_{\text{LUT}} \in \{1, 2\}$), with recall differences at most 10^{-4}. Both layouts are subspace-major. This



Figure 7.4. Cartesian table grouping at fixed $\alpha = 100$ (v71). Group counts describe g_{LUT}, not parallel scan groups. Curves are medians of per-dataset QPS ratios to the one-group arm; shading shows their range.

tests packing/load/unpack together; one aggregate rate per arm gives less timing evidence than v71's five rounds.

Figure 7.5 and table 7.4 then separate wider loads from lane cooperation in residual refinement. L0 is the original scalar path; L1 changes the load path without distributing a candidate across multiple lanes; L16 assigns 16 lanes to a candidate. The same process rotates arm order across five timing rounds, with three search calls per arm per round. For the twelve ordinary configurations, L16 improves end-to-end throughput by 1.015–1.789×. An additional arXiv configuration at $\alpha = 200$, explicitly a high-budget control, reaches 2.383×; it is not the gain range of the ordinary selected budgets.

The load-only control is bit-identical to L0 in the tested cases, yet already improves arXiv throughput by 1.368×. Lane cooperation adds a larger reduction in refinement time. On arXiv at $\alpha = 100$, the profiled refinement kernel falls from 4,277.5 to 716.8 μs, a 5.97× gain, while total throughput improves 1.789×. OpenAI-3072 gains 8.81× in refinement and 1.171× overall; routing, screening, selection and transfers remain.

For baseline wall-time fraction f and refinement speedup s, the replacement model gives

$$\frac{T'}{T} = (1 - f) + \frac{f}{s}, \qquad \frac{T}{T'} = \frac{1}{1 - f + f/s}.$$

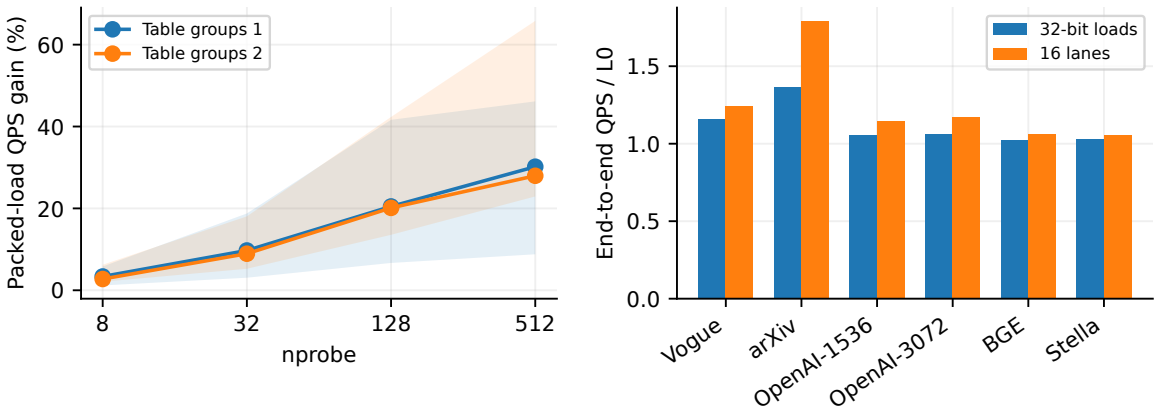


Figure 7.5. Packing and refinement controls. Left: median packed-load QPS gain across six datasets, with shaded minimum–maximum ranges, for each table grouping and probe count. Right: measured load-only and 16-lane end-to-end gains at 128 probes and the ordinary dataset-specific budgets. Panels use different experiment families.

Table 7.4. Refinement at 128 probes. Ratios are paired end-to-end gains over L0, except the last column, which is the profiled refinement-kernel gain. The kernel and application gains must not be interchanged.

Dataset	α	$L1/L0$ QPS	$L16/L0$ QPS	Kernel gain
arXiv	100	1.368	1.789	5.97
BGE	32	1.026	1.062	1.97
OpenAI-1536	8	1.055	1.145	3.34
OpenAI-3072	4	1.063	1.171	8.81
Stella	16	1.030	1.053	2.66
Vogue	64	1.161	1.240	3.72

ArXiv/OpenAI-3072 $f = 0.529/0.168$ predicts 1.787/1.175×, versus measured 1.789/1.171×. Separate profiling makes this explanatory, rather than a paired timing identity.

Lane FP32 reassociation reaches relative distance error $\approx 1.6 \times 10^{-6}$ and occasionally changes order. Sanitizer coverage is limited to recorded subsets; general bit identity is not claimed.

7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation

7.4.1 Strict CPU reference for computed-score selection

T4 sorts GPU-produced scores by $(D_{\mathrm{P}}, \mathrm{position})$ on CPU. The exporter follows scan layout/accumulation and matches valid returned scores bitwise; source comparison finds only export changes from the table-sweep version. Its 270 configurations cover Vogue/arXiv/OpenAI-3072, probes 1/8/128, $\alpha \in \{4, 100, 200\}$, $g_{\mathrm{scan}} \in \{0, 1, 2, 4, 8\}$ and both bound settings: 69,120 instances (table 7.5).

Table 7.5. T4 CPU-reference checks. Set and order equality use the complete primary key. Valid count checks $\min(c, n_c)$ survivors; final equality sorts retained computed refined scores by $(D_{\mathrm{H}}, \mathrm{ID})$, including their FP32 bits. Counts are query instances, with queries reused across configurations.

Path	Queries	Set / order exact	Tie-only	Valid count	Final exact
$g_{\mathrm{scan}} \geq 1$	55,296	55,296 / 55,296	0	55,296	55,296
$g_{\mathrm{scan}} = 0$	13,824	13,768 / 8,758	56	13,824	13,824

Grouped order matches in 55,296/55,296 instances. One-block distance order passes in 13,824/13,824, with 56 cutoff-tie set differences. All 19,120 underfull counts and 69,120 retained-score final selections pass. These test lemma 5.1 and propositions 5.1 and 5.2; separate scan/bound and sanitizer checks have their own coverage (section A.5).

7.4.2 Shared bounds preserve the selection target

Proposition 5.2 establishes the selection guarantee. The paired experiment measures the work saved by sharing that bound.

All 48 primary/final comparisons agree; throughput ratios are 0.994–1.562×. At arXiv $\alpha = 200$, batch 1,000 and eight groups, sharing cuts merge 2,629→471 μs and scan 7,968→5,513 μs. Some cells regress slightly.

The pilot's first group holds 46–81% of winners. Equal quotas need fallback; unequal ones leave a dominant group. Shared bounds avoid fixed quotas.

7.4.3 Exact probe selection and the limits of pipeline identity

The radix selector retains score arithmetic, including OpenAI TF32 (section 7.2.3). All 1,792 synthetic cases pass; probes and primary positions/distances match on all 1,000 queries of arXiv, Vogue and OpenAI-3072.

Table 7.6 shows end-to-end improvements of 1.034–1.167×. At batch 8, the profiled probe stage falls from 56.5 to 22.6 μs on arXiv, 52.7 to 14.7 μs on Vogue, and 77.8 to 16.7 μs on OpenAI-3072. Unchanged stages limit full-batch gains. These ratios use their own baselines.

Vogue's one-group/batch-8 probe/primary dumps match bytewise across binaries on all 1,000 queries, yet final recall is 0.9642/0.9644. The baseline itself gives 0.9642 in the identity run and 0.9644 in both timing runs. In that controlled case, variation arises after primary selection and without changing probes; the cause is unresolved. Scheduled/one-block outputs match within each checked process (appendix A).

Optional v70 FP16 matches v69 probe sets on 936/1,000 arXiv, 979/1,000 Vogue and 998/1,000 OpenAI-3072 queries; mean overlap is high. High overlap and small recall changes do not make the primitive exact. Those results are excluded from the main E1 frontier and from the exact-probe claim.

7.5 Held-out Budget Quality, Throughput and Payback

RQ3 compares the policy of chapter 6 with fixed $\alpha = 100$ and the stronger universal control $\alpha = 64$.

7.5.1 Unlabelled selection and a labelled diagnostic oracle

The six datasets are tested at 128 and 512 probes with the fixed, sampled, FULL, enumeration and oracle controls in section 6.4. The disjoint pools and 64 draws per sample size follow section 3.2.

All rule variants use equation (6.4) with $k = 10$ and $\epsilon = 1$. FULL uses all calibration queries; the diagnostic oracle uses held-out labels (table 6.1).

7.5.2 Full-pool calibration as the quality-first working point

Full-pool calibration uses all 500 calibration queries once per cell. Its selected arms meet the held-out 10^{-3} loss target in 12/12 cells; the largest observed loss is 0.0006. Their rates are 0.908–1.000 of the GT oracle's, and calibration costs 9.5–60.3 ms. The geometric-mean rate ratio to fixed 100 is 1.111×, or 1.193× over the ten cells where fixed 100 also meets the target. Table 7.7 shows the complete comparison. Against fixed 64, full-pool selection is faster in six cells, equal in two and slower in four: its main advantage is the observed coverage of all twelve quality targets.

These cells favour FULL for quality and $S = 128$ for calibration cost (section 6.2).

7.5.3 Lower-cost sampling and its quality risk

Table 7.8 reports the $S = 128$ result. The median selected budget matches the oracle budget in ten of twelve cells; it differs on arXiv at 128 probes and OpenAI-3072 at 512. Budgets span 4–200 across workloads. Median budget and mean recall need not describe one arm. Mean recall uses printed best recall minus mean loss, approximately to four decimals.

Table 7.6. v69/v65 end-to-end throughput ratios for the exact computed-score probe change. Each workload–batch comparison has ten timing rounds per binary, collected in alternating runs. Ratios > 1 favour v69.

Dataset	Batch 8	32	256	1,000
arXiv	1.074	1.063	1.035	1.034
Vogue	1.109	1.103	1.052	1.041
OpenAI-3072	1.167	1.117	1.055	1.050

Table 7.7. Full-pool calibration ($S = 500$). Loss is relative to the best measured held-out recall; rates are kQPS from the selected fixed arm. "Cal." is one full-pool calibration call per cell, not a repeated stream timing. Held-out counts are 500, or 499 for Stella.

Dataset	Probe	α_{full}	Loss	Q_{full}	vs 100	vs oracle	Cal. ms
Vogue	128	100	0.0000	202.9	1.000	0.908	10.1
arXiv	128	200	0.0000	151.5	0.728	1.000	10.0
OpenAI-1536	128	8	0.0000	267.7	1.599	1.000	9.5
OpenAI-3072	128	4	0.0000	145.0	1.691	1.000	17.1
BGE	128	64	0.0006	124.9	1.075	0.932	17.1
Stella	128	16	0.0006	110.4	1.153	1.000	20.6
Vogue	512	100	0.0002	100.1	1.000	0.969	20.9
arXiv	512	200	0.0000	69.0	0.831	1.000	25.1
OpenAI-1536	512	8	0.0000	114.9	1.326	1.000	21.8
OpenAI-3072	512	4	0.0002	52.1	1.219	0.977	40.5
BGE	512	64	0.0006	50.8	1.033	1.000	42.3
Stella	512	16	0.0006	36.0	1.041	1.000	60.3

Table 7.8. Held-out quality at $S = 128$. Rule α is the median across 64 draws; oracle α uses held-out labels. $\overline{R}_{\text{rule}}$ is reconstructed mean recall. "Losses" counts draws with recall loss $> 10^{-3}$ relative to the best tested held-out recall; L_{95} is the logged lower-order 95th-percentile loss estimate. R_{100} is fixed-budget recall.

Dataset	Probe	α: rule/oracle	R_{100}	$\overline{R}_{\text{rule}}$	Losses	L_{95}
Vogue	128	64 / 64	0.9680	0.9677	5/64	0.0022
arXiv	128	100 / 200	0.9748	0.9751	40/64	0.0024
OpenAI-1536	128	8 / 8	0.9280	0.9279	4/64	0.0000
OpenAI-3072	128	4 / 4	0.9252	0.9252	0/64	0.0000
BGE	128	32 / 32	0.9198	0.9190	1/64	0.0008
Stella	128	16 / 16	0.9892	0.9884	6/64	0.0030
Vogue	512	64 / 64	0.9916	0.9915	2/64	0.0002
arXiv	512	200 / 200	0.9878	0.9885	32/64	0.0016
OpenAI-1536	512	8 / 8	0.9740	0.9738	4/64	0.0000
OpenAI-3072	512	4 / 8	0.9704	0.9702	0/64	0.0002
BGE	512	64 / 64	0.9670	0.9661	24/64	0.0014
Stella	512	16 / 16	0.9954	0.9945	7/64	0.0030

ArXiv has 40/64 and 32/64 loss crossings at 128/512 probes, despite mean loss 0.0009. Fixed 100 loses 0.0012/0.0016: better than default need not meet the target. BGE/512 has 24 crossings and $L_{95} = 0.0014$; Stella reaches 0.0030; OpenAI-3072 has none (table 7.8). Four losses can coexist with reported $L_{95} = 0$ (section A.3).

7.5.4 Estimated throughput and calibration economics

With section A.3's mean selected-arm estimate, $S = 128$ beats fixed 100 in ten cells (1.028–1.691×). ArXiv gives 0.908/0.917 with better mean recall: no throughput payback.

Against fixed 64, sampling wins 9/12 cells: geometric mean 1.054×, range 0.821–1.367× (OpenAI 1.052–1.367×). Both fixed controls fail arXiv quality, preventing a uniformly quality-matched comparison.

Calibration costs 3.9–21.3 ms. Maximum positive estimated paybacks against fixed 100/64 are 0.54/2.72 s, or FULL 1.52/2.25 s (section 6.5, equation (6.15), and table 7.9). ArXiv controls fail quality; direct reuse below measures both cost and recall.

7.5.5 Complete decision traces and finite-pool risk

All 2,304 sampled and twelve FULL curves are monotone with enumeration-matched choices. Recorded subset sums reproduce 18,432 grid counts (equation (6.9)). This verifies recorded

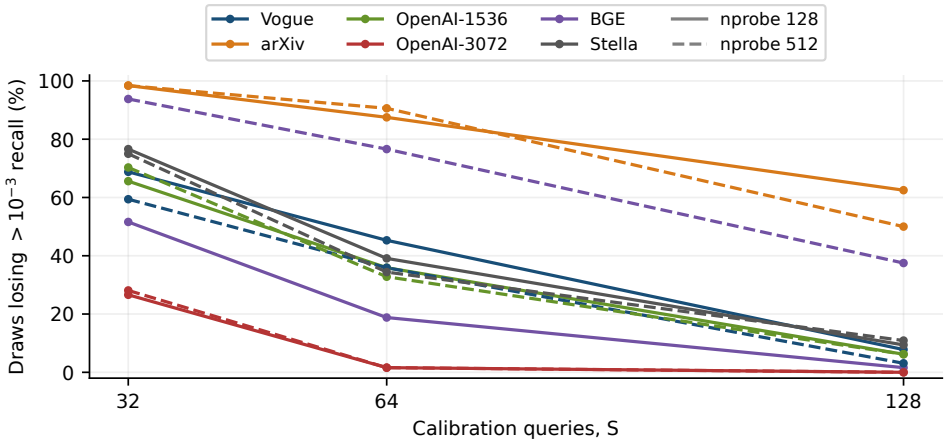


Figure 7.6. Held-out risk across sample sizes. The vertical axis is the percentage of calibration draws losing more than 10^{-3} held-out recall relative to the best tested budget. Increasing S also tightens the allowed slot-disagreement fraction because equation (6.4) keeps threshold one.

Table 7.9. Calibration economics, $S = 128$. Rates are kQPS and rule rate is mean selected-arm QPS. Ratios compare with fixed 100 and fixed 64. The "Batches" column gives payback relative to fixed 100 using 500 queries, or 499 for Stella; "Never" denotes no throughput payback. The quality column separately marks whether fixed 100 meets the held-out loss target.

Dataset	Probe	Q_{100}	$\overline{Q}_{\text{rule}}$	vs 100	vs 64	Cal. ms	Batches	Fixed 100 quality
Vogue	128	202.9	224.4	1.106	1.004	4.0	17.0	Pass
arXiv	128	207.9	188.8	0.908	0.821	4.2	Never	Fail
OpenAI-1536	128	167.5	268.5	1.603	1.367	3.9	3.5	Pass
OpenAI-3072	128	85.7	145.0	1.691	1.230	5.7	2.4	Pass
BGE	128	116.2	131.2	1.130	1.051	6.5	13.2	Pass
Stella	128	95.7	110.3	1.153	1.087	7.5	10.8	Pass
Vogue	512	100.1	102.9	1.028	0.996	7.5	55.4	Pass
arXiv	512	83.1	76.2	0.917	0.870	9.7	Never	Fail
OpenAI-1536	512	86.6	115.0	1.327	1.268	8.8	6.2	Pass
OpenAI-3072	512	42.8	52.1	1.219	1.052	13.1	6.2	Pass
BGE	512	49.2	51.1	1.039	1.006	15.8	41.2	Pass
Stella	512	34.6	36.0	1.041	1.028	21.3	37.1	Pass

aggregates; arbitrary subset/position invariance and general distance-only nesting remain conditional (section 6.3).

Maximum loss is 0.0050 (arXiv/512), 0.0046 (BGE/512) at $S = 128$, and 0.0294 at $S = 32$. FULL stays within 0.0006.

ArXiv/128 predicted/observed crossing rates are 0.965/0.984, 0.875/0.875, 0.620/0.625 for $S = 32, 64, 128$ (figure 7.7). The largest of 36 gaps is BGE/512: 0.144 (0.519/0.375). Mapping predictions to GT loss uses stable counts and held-out arm quality; calibration itself is label-free.

Ten alternating repeats support a low-budget timing plateau, rather than general runtime monotonicity.

7.5.6 Direct calibration-plus-reuse measurement

Four cells compare fixed 64/100, sampling and FULL across five rotated rounds of 500 repeated held-out batches. Timing includes gathering, calibration, budget installation and first graph setup; build/warm-up/scan calibration are excluded (table 7.10).

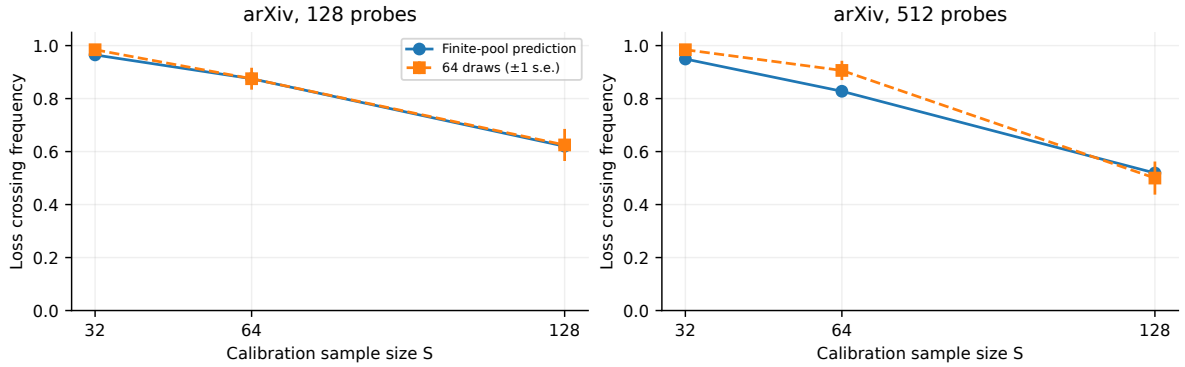


Figure 7.7. Finite-pool prediction and observed arXiv loss crossings. Bars show one estimated standard error over 64 independently drawn subsets conditional on the fixed pool, not uncertainty over future workloads.

Table 7.10. Direct total milliseconds for 500 reuse batches, averaged over five rounds, including budget calibration for sampled and FULL policies. Each batch has 500 queries. Quality is discussed with the timing results.

Dataset/probes	Fixed 100	Fixed 64	Rule $S = 128$	FULL
arXiv/128	1224	1102	1291	1680
OpenAI-1536/128	1491	1264	852	860
OpenAI-3072/128	2757	2160	1769	1783
BGE/512	5185	5018	4921	5062

OpenAI-3072/128 always selects 4: sampled/FULL totals are 1,769/1,783 ms versus fixed 100's 2,757 ms, all at recall 0.9252. OpenAI-1536 FULL takes 860 versus 1,491 ms at 0.9280; sampling takes 852 ms at 0.9276.

Direct savings start after 4/6 batches (OpenAI-1536), 4/9 (OpenAI-3072) versus fixed 100/64; fractional fits are separate (figure 7.8). BGE sampling takes 4,921 versus 5,185 ms at recall 0.9658/0.9670; FULL takes 5,062 ms at 0.9664. ArXiv FULL selects slower 200 to meet quality; fixed controls fail.

7.6 Batch Regimes, G Scheduling and Workload Skew

This section answers the performance-regime part of RQ2: when additional scan groups supply useful parallelism, and when their extra work dominates.

7.6.1 Parallelism versus repeated selection work

At batch 8, eight groups raise throughput 2.06–2.76×: arXiv scan drops 849→216 μs, with 16.7 μs merging. At batch 1,000 and $\alpha = 200$, independent grouping instead raises scan 3,530→7,652 μs plus 2,624 μs merging, reaching only 0.46× one-block QPS. Shared bounds reduce work, without removing this regime change.

7.6.2 A fixed rule evaluated on further configurations

The scheduler follows the fill/skew rule and pseudoquery calibration in section 5.5. Four configurations set the constants before the additional configurations were measured.

The means use thirteen small-batch configurations and eleven full-batch ones (Stella has 999 queries); held-out subsets use nine/seven (table 7.11). Eight groups give 0.886× at batch 1,000; worst regret is BGE's 1.217 at batch 256.

Vogue's CV 0.338–0.622 forces four groups at batch 1,000. Ratios span 0.895–1.175; $(n_{\text{probe}}, \alpha) = (16, 16), (16, 64), (32, 64)$ give 0.955, 0.895, 0.955. CV captures dispersion, not absolute work: the same rule enables the 0.97 lead but wastes work at low probes (section 7.2).

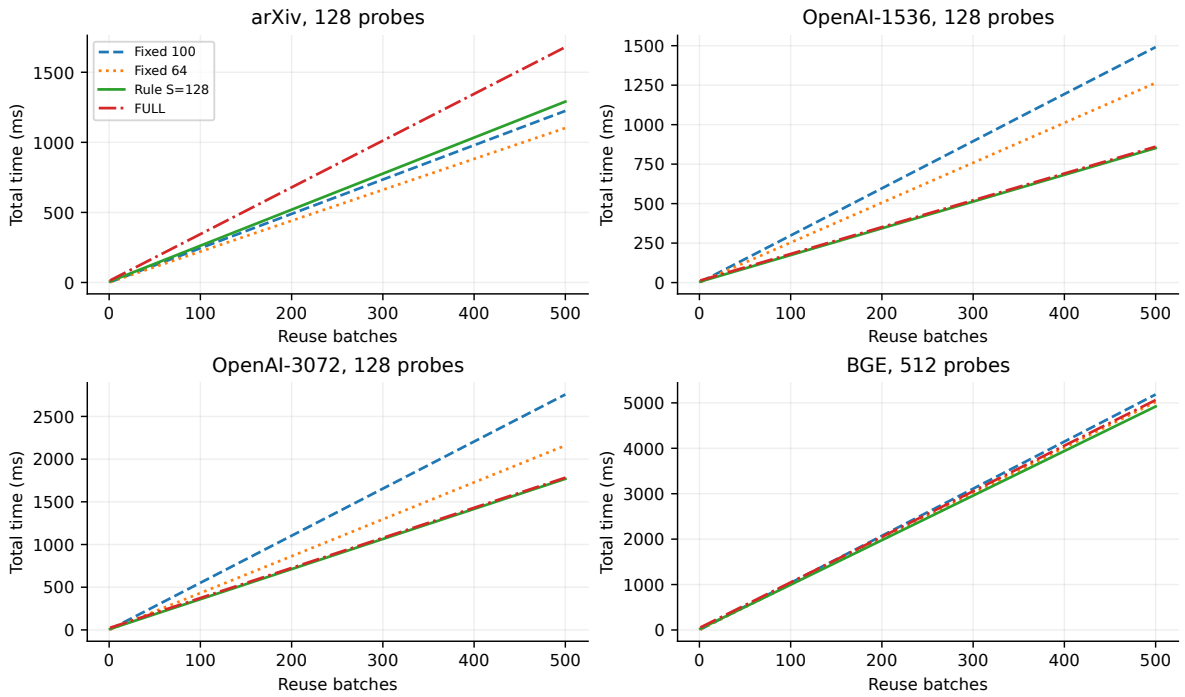


Figure 7.8. Direct mean cumulative time, including budget calibration and first-batch setup. Each curve replays one fixed held-out query pool. Policies can have different recall; timings alone are not a quality-matched ranking. Dataset/probe settings match table 7.10.

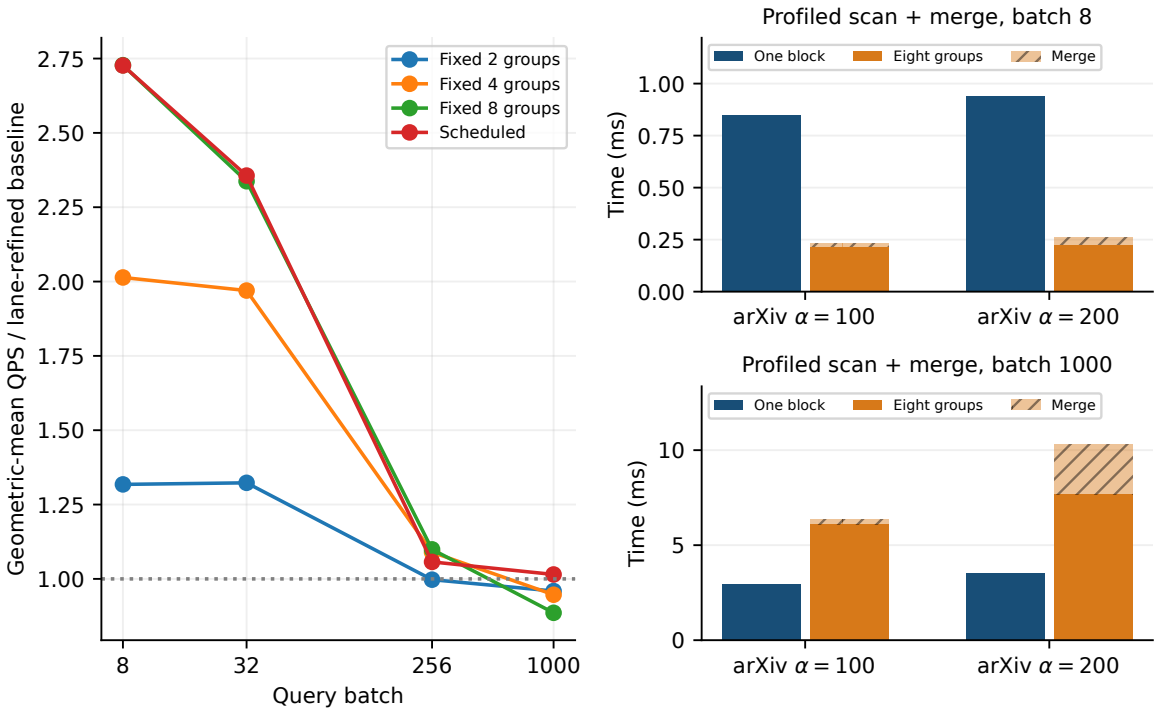


Figure 7.9. Execution regimes. Left: fixed shared-bound group counts and scheduled execution relative to the lane-refined one-block baseline (13 cells at small batches, 11 at batch 1,000). Right: paired scan and merge profiles at batches 8 and 1,000 for arXiv budgets 100 and 200. Hatching denotes merge time. The right panels use the independent-bound scan experiment; they explain a cost, not the scheduler's entire speedup.

Table 7.11. Integrated execution gains over v61 L16. Entries are geometric means of per-cell median paired speedups from five rounds. "Held out" excludes the four configurations used to set the rule; it means held-out configurations, not the budget experiment's held-out query split.

Batch	Cells	$g_s = 2$	$g_s = 4$	$g_s = 8$	Scheduled	Held out
8	13	1.318	2.014	2.728	2.727	2.930
32	13	1.323	1.970	2.337	2.356	2.560
256	13	0.997	1.090	1.099	1.057	1.041
1000	11	0.959	0.947	0.886	1.015	1.012

Across 250 variant/configuration pairs and 234,320 query comparisons, there are no non-tie primary differences or final-ID-set differences. Tied output order can differ. These are sequential synchronous throughput tests, not open-loop tail-latency tests.

7.7 Cold Construction, Resident Memory and Tested Scale

RQ1 is assessed through cold construction, resident memory and completed large-dataset searches; the E1 memory comparison also informs RQ4.

7.7.1 Cold builds and the lifecycle boundary

Three empty-cache repeats give Vogue 298–303 ms training and ≈ 0.56 s total, versus the slow archived 1.33 s. Stella takes 10.54–10.65 s versus archived 10.40 s (table 7.12).

Sections A.2 and 3.3 define startup/build boundaries; figure 7.10 compares repeated builds.

Stella takes 74.55/82.45 s with isolated CAGRA int8 (quantisation included), and median 86.59 s across three IVF-PQ configurations, versus JHQ's 10.54–10.65 s. Quality/resident memory are separate comparisons.

Five arXiv builds span 0.0012 recall. Stella startup takes 18.2–18.6 s with warm pages, 129.9 s on first read. Sampled 20 ms peaks are BGE 26.08 GiB and Stella 22.58 GiB. BGE's 12 GiB assignment buffers precede its 23.05 GiB gather payload, so they must not be summed (equation (4.13)).

7.7.2 Resident allocation and where compression helps

Figure 7.11 combines the component model in section 4.4 with measured resident totals. Its unattributed remainder is retained, and the preceding construction measurements give the sampled peaks.

Residuals occupy 80.7% of BGE's 11.93 GiB and 82.9% of Stella's 20.44 GiB totals, including context/workspace. Raw vectors need 38.50/67.81 GiB. Smaller α saves reads, not stored residual bytes (section 4.4).

Table 7.12. Cold JHQ construction and separately measured resident memory. Train/add are component medians of three supplementary normal builds; Total is the median paired sum. Memory comes from v71's two-group, 128-probe, $\alpha = 100$ runs and includes context and workspace. "Residual" is the modelled residual-code share of the measured used total, not the fraction read per query.

Dataset	Train s	Add s	Total s	Used GiB	Raw GiB	Residual
Vogue	0.30	0.26	0.56	1.47	2.67	45.4%
arXiv	0.48	0.51	0.99	2.58	6.45	62.5%
OpenAI-1536	0.61	0.38	1.00	2.39	5.72	59.7%
OpenAI-3072	1.16	0.67	1.82	4.14	11.43	69.0%
BGE	3.56	3.65	7.20	11.93	38.50	80.7%
Stella	3.55	6.99	10.55	20.44	67.81	82.9%

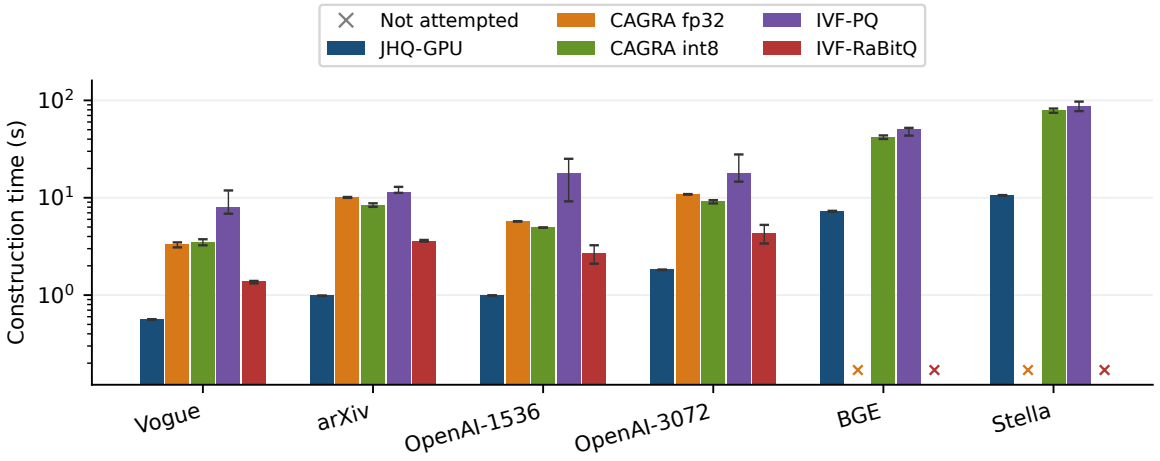


Figure 7.10. Recorded construction costs, with CAGRA int8 included. JHQ is the median and range of three supplementary normal cold builds with warm input pages; other bars are medians across successful measured index configurations, with minimum–maximum whiskers. Int8 includes dataset quantization; RaBitQ excludes the required serialization round trip. Missing large-dataset bars denote unattempted configurations, not measured build failures. The vertical axis is logarithmic.

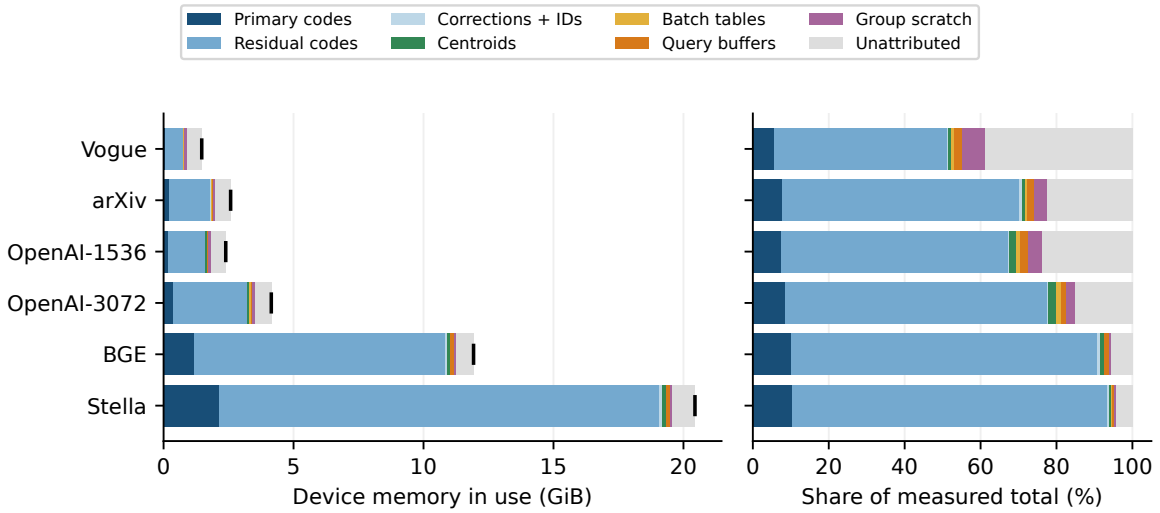


Figure 7.11. Resident-memory decomposition at two-group tables, 128 probes and $\alpha = 100$. Component sizes are modelled, markers are measured used totals, and hatching is the unattributed remainder. The totals include context and workspace. The preceding construction measurements give sampled peaks; these resident markers are not peaks.

The E1 measurements provide a separate cross-system view in table 7.13. Subtracting the baseline-measured 507.2 MiB context from JHQ's process total gives approximately 0.29–0.39 of CAGRA fp32's index allocation. JHQ's figure includes workspace, whereas CAGRA's is the build-time index delta. These are parameter-sweep ranges, not memory-matched recall points or a comparison of equal peak budgets. The CAGRA int8 allocations in the same table are of a similar order to JHQ's: compression relative to fp32 is shared by competing designs. The component model in table 4.1 explains this comparison.

7.7.3 Large-dataset feasibility and competitive regions

BGE contains 10,091,524 vectors and Stella 17,776,615, both with 1,024 dimensions and 32,768 IVF lists. Stella's queries are sampled from the corpus; its result therefore characterizes that benchmark rather than an independently collected external-query distribution. Figure 7.12 includes successful CAGRA int8 and IVF-PQ measurements. Full-precision CAGRA, RaBitQ configurations and FP32 brute force were excluded when computed storage exceeded capacity. These are planned exclusions; no out-of-memory or recall failure was measured.

CAGRA int8 fits Stella. On BGE it leads 1.35–2.96× at recall 0.88–0.95; JHQ leads 1.086× at int8's maximum 0.9579. Maximum BGE/Stella recalls are int8 0.9579/0.9783, JHQ 0.9663/0.9952, PQ 0.9525/0.9896.

Stella measured points give JHQ 185.4 kQPS/0.978679 recall and int8 66.3 kQPS/0.978278. Their 2.80× ratio is not recall-matched. Int8 lacks much of the upper JHQ coverage (appendix A).

7.8 Numerical Robustness, Anomalies and Recall Attribution

7.8.1 Quality resolution and repeated execution

A single GT hit changes Recall@10 by

$$\Delta r = \frac{1}{nk} = \begin{cases} 10^{-4}, & n = 1000, \\ 2 \times 10^{-4}, & n = 500. \end{cases}$$

Stella's $n = 499$ step is slightly larger. This is resolution, not uncertainty.

E1 retains 369 baseline repeat-ID flags, including tied reordering. These repeated-call checks differ from independent builds; the 401 convention/cross-protocol checks are in section A.2.

Near 0.97, unsnapped frontier slopes give 1.01–3.17% QPS change per 10^{-3} recall. Five-build ranges are Vogue 0.0008, arXiv 0.0012, OpenAI-1536 0.0004, OpenAI-3072 0.0002: few-percent local sensitivity, not a retrained-frontier bound. Cached-model FP32 checks retain OpenAI's advantage (section 7.2.3); arXiv/LUT near ties remain inconclusive.

RaBitQ's median five-round ranges are 3.79%, 3.34%, 2.87%, 5.25% of median QPS on Vogue/arXiv/OpenAI-1536/3072. These spreads are not confidence intervals. To reduce occasional 30–60 ms calls' influence, we recompute the 1,000-query rate as 10^6/ median(call ms). At recall 0.97, the resulting JHQ/RaBitQ ratios are 1.063, 0.995, 1.118 and 1.100. The three

Table 7.13. E1 allocation ranges. JHQ is process total minus a 507.2 MiB context estimate, including query workspace; both CAGRA columns are index allocations measured during build. Ranges cover the parameter sweep.

Dataset	JHQ net process GiB	CAGRA fp32 index GiB	CAGRA int8 index GiB
Vogue	0.90–1.01	2.89–2.91	0.89–0.91
arXiv	2.01–2.19	6.84–6.98	2.01–2.15
OpenAI-1536	1.79–2.33	5.95–5.97	1.67–1.68
OpenAI-3072	3.54–3.82	11.67–11.68	3.10–3.11

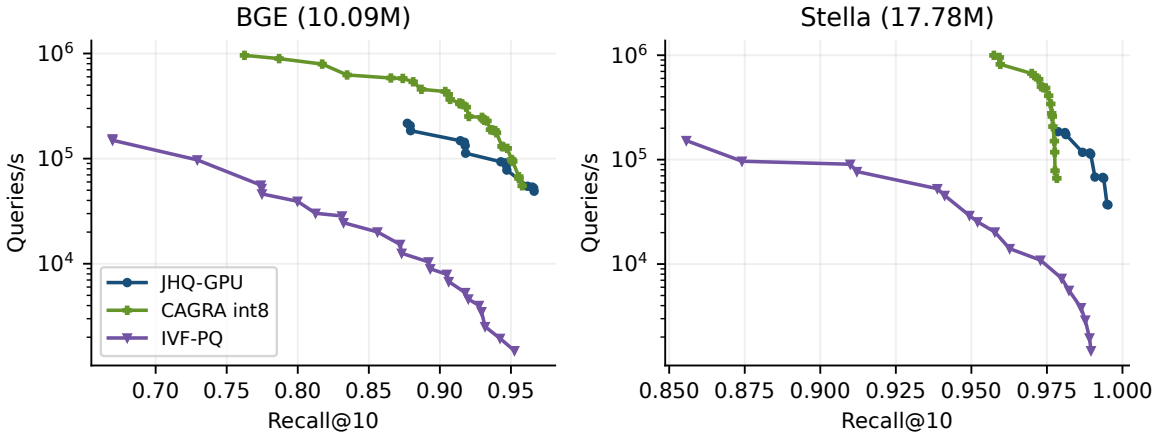


Figure 7.12. Large-dataset host-to-host frontiers, including the successful isolated CAGRA int8 reruns. BGE/Stella use 1,000/999 queries. Endpoints mark tested-grid coverage.

favourable directions and arXiv near parity remain, so the result is not solely a consequence of long calls.

7.8.2 Exactness is attached to a particular stage

The stage contracts in table 5.1 distinguish computed-score selection from raw-space kNN; lane rounding, cutoff ties and process variation qualify different stages.

The archived reconstruction pilot measures the remaining gap to full-probe reconstructed scoring. Across seven workload–budget arms of 128 queries each, scoring all probed candidates in FP64 improves recall over GPU outputs at selected budgets by 0–0.002344. For arXiv at 128 probes, $\alpha = 200$ matches the full-IVF reconstructed answer set on all 128 queries. At 100, the recall is 0.9734375 versus 0.9750 for full-IVF scoring: an additional 0.0015625 budget deficit beside the joint routing/reconstruction deficit of 0.0250 (section 6.2). These pilot aggregates include two budgets on one dataset–probe workload. FP64/GPU arithmetic differences prevent pure budget attribution in every arm. Separating routing/representation needs stage-wise GT coverage.

8 Discussion and Future Work

8.1 Three Cross-Layer Insights

8.1.1 A useful representation exposes a useful unit of reuse

Mailman loses despite greater reuse; two-group LUTs usually win (figure 7.4). Codebook quality must therefore be evaluated with achievable table placement/access cost (sections 5.2 and 5.3).

8.1.2 Useful parallelism is bounded by duplicated state and selection

With enough query parallelism, staging/selection/merging can dominate (figure 7.9). Vogue/BGE show missing absolute-work/decomposition features (sections 7.2 and 7.6). A richer scheduler must repay calibration; integrated gains do not isolate bounds.

8.1.3 Execution speed changes the value of avoiding refinement

Consider a latency model with fixed non-refinement cost $T_0 > 0$, high-/low-budget refinement times $F_h \geq F_\ell > 0$, and a common refinement-time multiplier $0 < \eta \leq 1$. The low-budget speedup and its derivatives are

$$S(\eta, T_0) = \frac{T_0 + \eta F_h}{T_0 + \eta F_\ell}, \tag{8.1}$$
$$\frac{\partial S}{\partial \eta} = \frac{T_0(F_h - F_\ell)}{(T_0 + \eta F_\ell)^2} \geq 0, \qquad \frac{\partial S}{\partial T_0} = \frac{\eta(F_\ell - F_h)}{(T_0 + \eta F_\ell)^2} \leq 0.$$

For $F_h > F_\ell$, faster refinement reduces the value of cutting its budget; lower fixed cost increases it. The derivatives hold with non-refinement work fixed. Budget-dependent scan maintenance and resource transitions require the controlled comparisons of sections 7.5 and 7.5.6. Quality and reuse remain constraints (table 7.7 and figure 7.6).

8.2 Two Deep Negative-Result Case Studies

8.2.1 Mailman-style shared sums: arithmetic reuse meets scatter cost

Mailman reuses finite-alphabet matrix–vector sums [20]. We test atomic buckets, private histograms and widths 4/6/8 against packed two-table scoring on frozen Vogue/arXiv/OpenAI-3072 streams, 128 probes, batches 8/1,000. Codes and mathematical scores stay fixed.

The best alternatives reach only 0.187–0.581 of baseline throughput (figure 8.1). Bucket accumulation dominates profiles. List bias repeats destinations; private histograms need more state/shared updates. Narrower groups sometimes help, but no tested variant wins. We stopped these implementations, leaving other reuse designs open.

The pilot emits scores, so FP32/FP64 cutoff differences do not establish neighbour-pipeline equivalence. For scorer ratio s and replaceable fraction f,

$$T'/T = (1 - f) + f/s.$$

This is a replacement model, not an end-to-end measurement: integration stopped. The dictionary study stopped at structural statistics; neither contributes a system-frontier point.

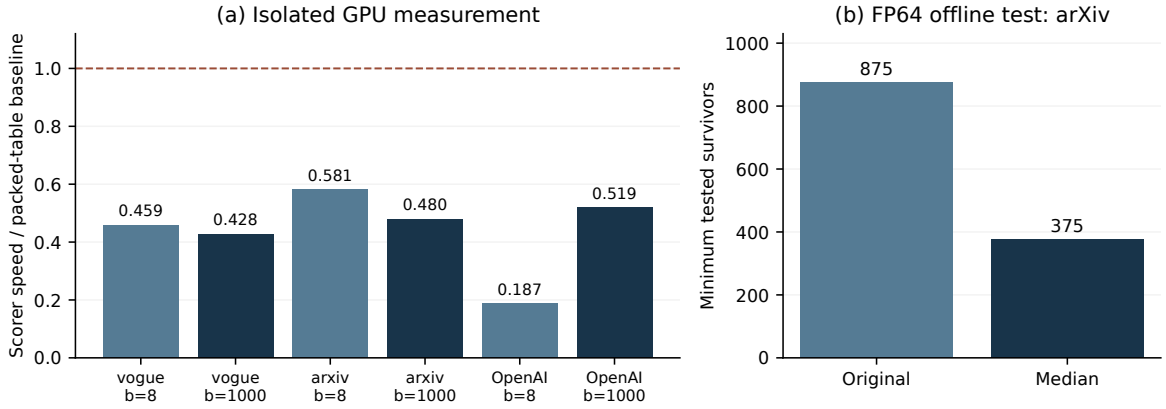


Figure 8.1. Two distinct evidence boundaries. Left: best measured Mailman scorer / packed-table scorer speed on an RTX 5090, with Vogue, arXiv and OpenAI-3072 at two batches and 128 probes; all ratios are below one. Right: minimum survivor counts meeting the original JHQ recall target on arXiv's 400-query FP64 offline test split. The candidate reduction is not a GPU speedup.

8.2.2 Rank-oriented coding: a local gain is not a general replacement

The ranking replay uses FP64 exported candidates and 400/200/400 calibration/validation/test queries. It tests weighted scores, median/list thresholds, learned losses and linear-MMSE decoding. Lower reconstruction/inner-product MSE can worsen neighbour order; changed primary levels also move residual reference points.

ArXiv median thresholds reduce the minimum tested qualifying count from 875 to 375; two other workloads fail the criterion. Rotation preserves corpus-mean norm, leaving bias despite Gaussian intuition. Centring must also preserve the refined-score ceiling; this gain is workload-specific.

A GPU follow-up imports median thresholds/levels and retrains residuals. At arXiv's original $\alpha = 100$ target it gains 1.40/1.21× at batches 8/1,000. Vogue reaches only 0.9630 versus target 0.9644; OpenAI-3072 ties. Three-round results use separate states/queries outside E1.

8.3 Threats to Internal and External Validity

Table 8.1 consolidates threats to the main interpretations. Controls are in Chapter 3; repeat/build/precision checks are in section 7.8.

Table 8.1. Threats that can reverse or narrow a conclusion.

Claim at risk	Alternative or reversal	Control and remaining test
Execution gain	Changed scores, ties or index state explain an apparent speedup	Strict T4 CPU sorting and table-form diagnostics; finite coverage, with candidate dumps no longer retained
Budget quality	Finite-pool sampling or drift hides damaging queries	Disjoint GT evaluation, full traces, subset-sum checks and FULL enumeration; future-query and arbitrary-subset invariance remain untested
Competitive margin	Grid density, call tails or API overhead changes the ordering	Both recall estimators, retained slow calls and int8 baseline; TF32/FP32 sensitivity; T5 exposes small-batch reversals; cross-device comparisons remain unmeasured
Capacity/build claim	Transient allocations exceed a resident sample	Phase bytes, repeated empty-cache builds and 20 ms device peaks; sub-interval allocations and full retrained frontiers remain unmeasured

Coverage is one GPU, squared-L2 top-10 and six prepared datasets (section 7.2.5). Near duplicates may remain; relevance labels are absent. Updates, concurrency, drift, service latency, billion-scale and multi-device use are untested.

8.4 Testable Research Directions

8.4.1 Risk-controlled agreement with a reuse-aware objective

A future cost-aware policy needs valid query-level agreement bounds, matched-regime counts, a fixed reference, sampling/order assumptions and drift-triggered renewal. Absolute GT targets additionally require labels or a justified reference-to-GT link.

Compare fixed sampling, reusable rank curves and cost-aware choice on held-out pools using total time/tail agreement. A four-cell sequential controller lost to extra launches. Success requires lower total cost than a quality-matched control while meeting the risk bound.

8.4.2 Capacity-aware residual placement

A future residual cache could retain primary codes on-device and fetch missing residual blocks from host memory. Fix codes, routing, survivors and arithmetic; vary device cap, batch and locality. Measure bytes, hits, overlap, tail latency and QPS. Lowering α alone does not extend this storage capacity.

8.5 Conditional Deployment Recommendation

At recall 0.97, E1 supports batch-1,000 throughput; T5 favours CAGRA fp32 on every batch-8 dataset. Batch-32 OpenAI-3072 changes near-tie ranking with the estimator (section 7.2.5). Start with FULL, then validate deployment quality, reuse cost and memory (section 8.3).

9 Conclusion

9.1 Answers to the Four Research Questions

RQ1: Construction and capacity. GPU training, batched encoding, IVF placement and packing build the resident hierarchy. The largest test holds 17.78 million vectors at 20.44 GiB, built in 10.54–10.65 s (section 7.7). Sampled peaks are Stella 22.58 GiB and BGE 26.08 GiB at add entry.

RQ2: Correct and efficient execution. Cartesian LUT splitting is exact in real arithmetic; complete-key retention/merging and witnessed bounds preserve grouped selection under the stated conditions. Two-group tables lead in 22/24 cells. Against lane refinement, scheduling gives geometric-mean 2.727× at batch 8 and 1.015× at 1,000: benefits depend on batch/work distribution (sections 7.3 and 7.6). T4's strict CPU reference matches all 55,296 grouped instances in complete-key order; 56 one-block set differences are exclusively cutoff ties (section 7.4.1).

RQ3: Refinement budget and quality. FULL calibration meets the tested relative recall-loss target in 12/12 cells, with maximum observed loss 0.0006. At $S = 128$ reaches 40/64 crossings in one cell (section 7.5). All FULL choices match enumeration, but ten distance-only cells lack a general nesting guarantee. Reference agreement is not population GT recall. Four direct reuse tests show workload-dependent quality/time gains.

RQ4: Competitive operating regions. At batch 1,000/recall 0.97, JHQ leads on three datasets and nearly ties RaBitQ on arXiv; Vogue's lead needs grouping (section 7.2). OpenAI E1 uses TF32. Cached-model FP32 checks retain advantages over archived D baselines and contemporaneous T1 anchors, without FP32 retraining (section B.4.1). Int8 wins lower targets; RaBitQ wins some higher ones. CAGRA fp32 wins all batch-8 datasets; batch-32 near ties depend on the estimator (section 7.2.5).

9.2 Closing Principle and Outlook

Representation, execution and effort must share a quality target. Next steps are reusable risk-controlled calibration and capacity-aware residual placement (section 8.4).

Bibliography

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43 (5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2022. URL `https://arxiv.org/abs/2110.01052v5`. Version 5; Section 2.3.1.

[4] Anthropic. Claude: Implementation, experiment and review assistance record. Project AI-use record, 2026. Implementation and experiment records; original prompts and interaction metadata unavailable.

[5] Tony Assi. Vogue933k embeddings. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/tonyassi/vogue933k-embeddings`. Upstream attribution recorded in the project data preparation source.

[6] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3 (4):242:1–242:26, 2025. doi: 10.1145/3749160. URL `https://arxiv.org/abs/2505.19001`.

[7] Artem Chirkin, Akira Naruse, Tamás Fehér, Yong Wang, and Corey Nolet. Accelerating vector search: NVIDIA cuVS IVF-PQ part 2, performance tuning, 2024. URL `https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-performance-tuning-part-2/`. 18 July; accessed 28 October 2026.

[8] FAISS contributors. FAISS clustering parameters, version 1.9.0. `https://github.com/facebookresearch/faiss/blob/v1.9.0/faiss/Clustering.h`, 2024. Default minimum-points warning threshold; accessed 28 October 2026.

[9] FAISS contributors. *Index I/O, Cloning and Hyper-parameter Tuning*, n.d.. URL `https://github.com/facebookresearch/faiss/wiki/Index-IO,-cloning-and-hyper-parameter-tuning`. Auto-tuning the runtime parameters; accessed 28 October 2026.

[10] FAISS contributors. *IndexRefine: Candidate Expansion and Refinement*, n.d.. URL `https://faiss.ai/cpp_api/file/IndexRefine_8h.html`. C++ API documentation; accessed 28 October 2026.

[11] Jianyang Gao and Cheng Long. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2405.12497`. SIGMOD 2024 paper.

[12] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2409.09913`.

[13] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization for approximate nearest neighbor search. In *IEEE CVPR*, pages 2946–2953, 2013. doi: 10.1109/CVPR.2013.379. URL `https://openaccess.thecvf.com/content_cvpr_2013/html/Ge_Optimized_Product_Quantization_2013_CVPR_paper.html`.

[14] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*, pages 331–340, 2012. doi: 10.1145/2304576.2304621. URL `https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf`.

[15] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020. URL `https://proceedings.mlr.press/v119/guo20h.html`.

[16] Jiabao Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[17] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[18] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL `https://arxiv.org/abs/1702.08734`.

[19] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020. doi: 10.1145/3318464.3380600. URL `https://www.pdl.cmu.edu/PDL-FTP/BigLearning/mod0246-liA.pdf`.

[20] Edo Liberty and Steven W. Zucker. The Mailman algorithm: A note on matrix–vector multiplication. *Information Processing Letters*, 109(3):179–182, 2009. doi: 10.1016/j.ipl.2008.09.028. URL `https://www.cs.yale.edu/publications/techreports/tr1402.pdf`.

[21] Rui Luo and Mengxuan Zhang. JHQ-GPU: Representation-driven GPU execution and workload-adaptive hierarchical refinement. Project manuscript source, 2026. Project manuscript.

[22] National Institute of Standards and Technology. *Dataplot: Exact Binomial Confidence Limits*, n.d.. URL `https://itl.nist.gov/div898/software/dataplot/refman2/auxillar/exacbino.htm`. One-sided exact binomial limits; accessed 28 October 2026.

[23] National Institute of Standards and Technology. *Dataplot: Hypergeometric Probability Density Function*, n.d.. URL `https://www.itl.nist.gov/div898/software/dataplot/refman2/ch8/hyppdf.pdf`. Sampling without replacement; accessed 28 October 2026.

[24] NVIDIA. *NVIDIA Blackwell Tuning Guide*, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/blackwell-tuning-guide/index.html`. Section 1.4.1.1; accessed 28 October 2026.

[25] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html`. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[26] NVIDIA. *PTX ISA, Version 9.0*, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/parallel-thread-execution/index.html`. Sections 8.1, 8.4.2, 8.7 and 8.10.3; accessed 28 October 2026.

[27] NVIDIA. *CUDA Programming Guide: The CUDA Platform*, 2026. URL `https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html`. PTX compatibility and just-in-time compilation; accessed 28 October 2026.

[28] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[29] OpenAI. ChatGPT: Project review assistance record. Project AI-use record, 2026. Retained review excerpts; original prompts and interaction metadata unavailable.

[30] OpenAI. Codex: Selected thesis-preparation interactions. Project AI-use record, 2026. Selected prompts and uses; compiled 29 October 2026. Partial record; historical interaction metadata unavailable.

[31] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 1536 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M`. Accessed 28 October 2026.

[32] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 3072 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M`. Accessed 28 October 2026.

[33] Jifan Shi, Jianyang Gao, James Xia, Tamás Béla Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

[34] The Alexandria Index. arXiv abstracts dataset. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/macrocosm/arxiv_abstracts`. Accessed 28 October 2026; exported embedding preparation is project-specific.

[35] The Information Engineering Lab. Stella TREC24 BioGen embedding. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/ielabgroup/stella_trec24_biogen_embedding`. Corpus split; accessed 28 October 2026.

[36] Upstash. Wikipedia 2024-06 BGE-M3. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/Upstash/wikipedia-2024-06-bge-m3`. Italian configuration; accessed 28 October 2026.

[37] Karthik Venkatasubba, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. BANG: Billion-scale approximate nearest neighbour search using a single GPU. *IEEE Transactions on Big Data*, 11(6):3142–3157, 2025. doi: 10.1109/TBDATA.2025.3581085. URL `https://ssomesh.github.io/docs/papers/tbd25.pdf`.

[38] Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on GPU: A comprehensive study and new methods. In *SC '23*, 2023. doi: 10.1145/3581784.3607062. URL `https://sc23.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap294.html`.

[39] Weijie Zhao, Shulong Tan, and Ping Li. SONG: Approximate nearest neighbor search on GPU. In *IEEE ICDE*, pages 1033–1044, 2020. doi: 10.1109/ICDE48307.2020.00094. URL `https://www.cs.rit.edu/~wjz/papers/conference/2020-icde-song.pdf`.

A Reproducibility and Experiment Manifest

This appendix specifies the experimental environment and measurement protocol. Section B.5 lists available materials.

A.1 Source, Environment and Trained States

Table A.1 records version evidence. Chapter 3 gives environment/JIT settings; historical binary/model fingerprints are incomplete.

Table 3.2 defines data/GT. Sweeps reuse caches; cold builds do not. Isolated large-baseline reruns replace contaminated builds. Stella uses 999 queries; skipped batch-1,000 runs contribute no points.

A.2 Measurement Contracts

E1 takes median QPS across five rounds, normally 24 calls each. Warm-up reduces calls at 274 points; its initial conversion/validation interval differs from steady-state timing. Timers include host staging, int8 quantisation and transfers despite API/output-memory differences. Device-only timing is auxiliary.

The 369 repeat-ID flags include tied reordering; 401 adds cross-protocol disagreement. Flags do not exclude points; JHQ lacks this flag. Cold timers exclude constructor/host QR/warm-up. Build peaks include context, sampled every 20 ms; synchronised one-buffer diagnostics are separate.

A.3 Budget Harness Contract

The campaign replaces the fallback 100-start grid with 200-start. Chapter 3 defines pools/draws. Initial warm-up visits all queries without supplying GT/outputs to agreement. Scan calibration uses separate database pseudoqueries.

Fixed arms use one warm-up plus three timed calls. Calibration includes searches and $O(Sk^2)$ host comparison, bypassing capture and excluding gathering/scan calibration. p95 uses index $\lfloor 0.95(64-1) \rfloor$. Callers apply the returned index with `set_alpha(A[index])`. Original selected-arm rates reuse fixed-arm timings.

Enumeration matches all 2,304 sampled and twelve supplementary FULL choices; subset sums match 18,432 grid counts. General order/invariance premises remain in section 6.3 and equation (6.9). GT-risk mapping uses held-out labels.

A.4 Supplementary Evidence

A1 separates first-read, warm-input and synchronised builds. C1 audits four layouts/sample recodings; FP64 norms use stored rotations, not GPU-rotated outputs. A3 checks all OpenAI queries and 10,299 base rows without entity IDs.

Table A.1. Recorded component provenance. Report and script identities are distinguished from raw run headers.

Component	Recorded source evidence
E1 final pipeline	Environment logs include `a31c22d`; extensions retain separate metadata
Lane refinement (v61)	Raw header `b37efa4`
Grouped scan (v62)	Report `862543a`; run script `e0d0518`
Shared bound (v63)	Source and build flags archived; no inferred run head
Batch scheduling (v65)	Raw header `d425ab8`
Radix probe (v69)	Comparative runs and source archived; binary SHA unrecorded
Budget / table sweep	Raw header `d5e1526`
Large int8 rerun	Stella degree-64 metadata `ad433a4`
Historical budget	Raw header `bf31b41`, v57 family

A2 reassembles extracted sm_80 PTX with CUDA 13.0.88 `ptxas` for sm_120. Aligned eight-byte accesses support the PTX contract; toolkit SASS is not a capture of driver-JIT code.

D varies add/query GEMMs on TF32-trained caches against archived baselines. T1 separately adds same-session anchors (section B.4.1); neither retrains FP32 models.

B reproduces `srand(12345)` choices. Direct reuse uses a separate seed, five rotated-policy rounds and 500 repeated batches. Timing includes gathering, calibration, budget installation and first-graph setup; build/scan calibration/warm-up are excluded. Schedule startup is separate.

A.5 Strict Selection, Small Batches and Artifact Access

T4 v73 adds two export blocks to v71. CPU sorting uses GPU-produced scores; bit identity covers valid returned primary entries, and padding checks valid counts. Table-form checks use 128 probes, $\alpha = 100$, 256 queries, shared bounds, $g_{\text{scan}} \in \{0, 2\}$, $g_{\text{LUT}} \in \{1, 2, 4, 8\}$. Summaries retain counts after candidate-dump deletion. Empty/adversarial lists and cross-process Vogue variation remain separate questions.

T5 uses fixed synchronous batches and five host-to-host rounds, including null calls/int8 query quantisation. JHQ/CAGRA reuse E1 grids; RaBitQ uses a separate sweep. Both estimators retain qualifying endpoints. APIs/warm-ups differ; concurrency is untested.

GPU reproduction needs original vectors, matching states/builds and the specified environment; section B.5 lists availability.

B Supplementary Results and Correctness Checks

Parameter coverage/checks follow; Appendix A defines protocol.

B.1 Parameter Coverage

E1 has 1,590 valid configurations (table B.1). Values below are observed sets, not necessarily full Cartesian grids.

Table B.1. Exact E1 configuration counts, including coverage extensions. Counts are parameter points, not independent index builds.

Family	Vogue	arXiv	OpenAI-1536	OpenAI-3072	Total
JHQ-GPU v69	27	29	37	39	132
CAGRA fp32	64	64	64	64	256
CAGRA int8	64	64	64	64	256
IVF-RaBitQ	132	132	264	198	726
IVF-PQ	54	54	54	54	216
GPU brute force	1	1	1	1	4
Total	342	344	484	420	1590

JHQ-GPU. All E1 points use $B = B_r = 8$, $M = d/8$, $k = 10$ and batch 1,000. Vogue uses 4,096 IVF lists; the other three use 8,192. Vogue observes probes 16/32/64/96/128/192/256/384/512 and $\alpha \in \{16, 32, 64, 128\}$ over 27 tuples. ArXiv adds probe 1,024 and uses $\alpha \in \{25, 50, 100, 200\}$ over 29 tuples. Both OpenAI grids use probes 32/64/128/256/512/1,024; their observed multipliers are respectively $\{4, 8, 16, 32, 64, 128, 256, 512\}$ and $\{2, 4, 8, 16, 32, 64, 128, 256\}$, over 37 and 39 tuples. The reported scheduled path uses four groups on Vogue and one block elsewhere. OpenAI uses TF32 rotation/centroid GEMMs in historical v69; the v72 FP32 sensitivity is a separate family (section 7.2.3).

CAGRA fp32 and int8. Each representation has 64 points per dataset: graph degrees 32/64, intermediate degree 128, `ivf_pq` construction, `itopk_size` in 32/64/96/128/192/256/384/512 and search width 1/2/4/8. Int8 records quantile 0.99 for dataset scaling and includes query quantisation in the host search boundary.

IVF-PQ. Eight-bit subcodes use 192/384/768-byte codes at dimension 768, and 384/768/1,536-byte codes on OpenAI. Each length has fp16/fp32 lookup tables and probes 16/32/64/96/128/192/256/384/512: 54 points per dataset. Refinement ratio is one; list counts match JHQ, while code lengths/memory differ.

IVF-RaBitQ and brute force. RaBitQ uses 4/8/9 bits, probes 16/32/64/96/128/192/256/384/512/768/1,024 and modes LUT16/LUT32/QUANT4/QUANT8 (no LUT16 on OpenAI-3072). OpenAI adds 4,096-list indexes to the 8,192-list default; the registry records combinations. Each dataset has one FP32 squared-L2 brute-force point. Untested coverage is not a failure.

Auxiliary grids. LUT tests cover six datasets, probes 8/32/128/512, $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ and $\alpha = 100$: 24 cells/96 arms. Budget tests use probes 128/512, $A = (200, 100, 64, 32, 16, 8, 4, 2)$, 64 draws per $S \in \{32, 64, 128\}$ and $\epsilon = 1$. FULL uses all 500 calibration queries. Held-out counts are 500 (Stella 499). Boundaries are specified in sections A.3 and A.4.

B.2 Correctness and Numerical Evidence

Table B.2 separates numerical, algorithmic and memory evidence. Chapter 5 proves order for computed scores under explicit contracts; recall evaluates raw-vector recovery.

Table B.2. Validation coverage and the object actually checked.

Boundary	Completed evidence and scope
Representation	Four all-row offset/ID audits; sampled raw-input FP64 recoding and stored-matrix geometry; TF32 deviations identified
Cartesian LUTs	Real-arithmetic factorisation; 96 timed arms; T4 per-candidate FP32 error and ranking diagnostics across four table forms
Global selection	T4: 270 configurations, 69,120 query instances; 55,296 grouped instances match strict CPU complete-key sorting in order
Shared bound	48 paired checks; extracted scalar 64-bit PTX and toolkit SASS support whole-key access; actual driver SASS not captured
Scheduling	250 variant/configuration and 234,320 query comparisons, retaining distinctions between ID sets and tie order
Probe selector	1,792 synthetic cases with zero failures; real-query ID/offset checks, relative to computed routing scores
Budget search	2,304 monotone sampled curves and 12 FULL enumeration matches; 18,432 sampled grid counts reproduce full-pool subset sums

The smaller representation-audit databases have identical primary and residual bytes across one-/two-pass builds when joined by ID. Corrections differ by at most 8.94×10^{-8} and 1/1/40 rows change list assignment; Stella validates its natural two-pass path (section 4.6.2). T4 supplies a strict computed-score reference (section A.5). A controlled same-model cross-process Vogue diagnosis remains undone: matching archived probe/primary dumps accompany recall 0.9642/0.9644. Within-run checks do not explain this anomaly.

B.3 Secondary Results and Evidence Status

Adverse regimes. Worst scheduling regret is 1.217 at batch 256; Vogue E1 ratios span 0.895–1.175 (section 7.6). T5 CAGRA fp32 wins all batch-8 datasets at 0.97; batch-32 near ties depend on the estimator (section 7.2.5).

B.4 Construction Capacities and Precision Anchors

Table B.3 separates staging/assignment capacities: three buffers, 65,536 add rows, 128 MiB per input/rotation array. These are neither active rows nor process peaks.

Table B.3. Default construction-buffer capacities. Assignment scratch is $4n_b n_{\text{list}} a_{\text{assign}}/2^{30}$ GiB, using a_{assign} rather than a_{in}.

d	n_{list}	a_{in}	a_{assign}	Dots GiB
768	4,096	43,690	8,192	0.375
1,024	32,768	32,768	32,768	12.000
1,536	8,192	21,845	32,768	3.000
3,072	8,192	10,922	32,768	3.000

B.4.1 Contemporaneous precision anchors

T1 remeasures recall-0.97 anchors in one session with E1 pools. Twelve pairs per OpenAI dataset bracket baselines with two five-round JHQ repeats; QPS combines medians geometrically. Mean FP32/TF32 ratios are 0.956/0.964, changing add/query arithmetic on TF32-trained caches.

Table B.4. T1 at recall 0.97 and batch 1,000. Baseline ratios use interpolation / strict threshold; FP32/TF32 uses interpolation. These same-session anchors are separate from D's cross-session comparisons.

Dataset	FP32/TF32 at 0.97	FP32 / RaBitQ	FP32 / CAGRA
OpenAI-1536	0.980	1.126 / 1.053	1.752 / 1.633
OpenAI-3072	0.973	1.328 / 1.316	1.825 / 2.019

FP32 remains numerically ahead under both estimators. OpenAI-3072's RaBitQ bracket (0.9669–0.9796) shows grid sensitivity. T1 neither replaces E1 nor retrains FP32 models.

B.5 Artifact Access

The project archive holds source/measurement snapshots and selected T4/T5, T1, A2 inputs. Reproduction additionally needs unavailable complete vectors, states, historical executables and the matching environment.